



InfoBus 1.2 Specification

The widespread adoption of the Java programming language by the Internet community creates an opportunity for developers to create a new class of interactive applications. The language and environment specifications provide mechanisms for the creation and management of small reusable feature sets known as Java Beans, whose functions generally represent only a portion of a Web application. However, the specifications do not suggest methods by which these beans should dynamically exchange data.

This work offers a solution to the problem of interconnecting beans by defining a small number of interfaces between cooperating beans, and specifying the protocol for use of those interfaces. In this specification, the fundamental building block for data exchange is the "data item." The specification details how various forms of information are represented as data items, the lifetime management of these items, and the protocols for querying and extracting information from these items.

The protocols described here are based on the notion of an info bus. That is, all components which implement these interfaces can plug into the info bus. As a member of the bus, any component can exchange information with any other component in a structured way. Generally, the bus is asynchronous and is symmetric in the sense that no component may be considered the master of the bus. However, provision is made in the protocol for a controlling component that can act as the bus master or arbitrator of bus conversations.

InfoBus 1.2 adds a new access interface, `ReshapeableArrayAccess`, that permits consumers to change dimensions of, or insert or delete rows or columns of an array data item.

© 1999 Sun Microsystems, Inc., 901 San Antonio Road, Palo Alto, California 94303, U.S.A. All rights reserved.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the United States Government is subject to the restrictions set forth in DFARS 252.227-7013(c)(1)(ii) and FAR 52.227-19. The release described in this document may be protected by one or more U.S. patents, foreign patents, or pending applications. Sun Microsystems, Inc.

(SUN) hereby grants to you a fully-paid, nonexclusive, nontransferable, perpetual, worldwide limited license (without the right to sublicense) under SUN's intellectual property rights that are essential to practice this specification. This license allows and is limited to the creation and distribution of clean room implementations of this specification that:

- (i) include a complete implementation of the current version of this specification without subsetting or supersetting;
- (ii) implement all the interfaces and functionality of the standard java.* packages as defined by SUN, without subsetting or supersetting;
- (iii) do not add any additional packages, classes or methods to the java.* packages;
- (iv) pass all test suites relating to the most recent published version of this specification that are available from SUN six (6) months prior to any beta release of the clean room implementation or upgrade thereto;
- (v) do not derive from SUN source code or binary materials; and
- (vi) do not include any SUN binary materials without an appropriate and separate license from SUN.

Sun, Sun Microsystems, Sun Microsystems Computer Corporation, the Sun logo, the Sun Microsystems Computer Corporation logo, Java, JavaSoft, JavaScript, and HotJava are trademarks or registered trademarks of Sun Microsystems, Inc. UNIX® is a registered trademark in the United States and other countries, exclusively licensed through X/Open Company, Ltd. All other product names mentioned herein are the trademarks of their respective owners.

THIS PUBLICATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. THIS PUBLICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN; THESE CHANGES WILL BE INCORPORATED IN NEW EDITIONS OF THE PUBLICATION. SUN MICROSYSTEMS, INC. MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THIS PUBLICATION AT ANY TIME.

Contents

Preface	5
1. Overview	6
1.1 Design of data flow between components	6
1.2 Component types	6
1.3 Principal requirements for the InfoBus	6
1.4 The InfoBus protocol for data exchange	7
1.5 The nature of data items	7
2. Membership	9
2.1 InfoBus instances	9
2.2 Details of the membership process	9
2.3 Revoking data items on changes to membership	11
2.4 InfoBus naming conventions	11
2.5 Security considerations for membership	11
2.6 Monitoring member departures	11
2.7 About “Stale” InfoBus Instances	12
2.8 The InfoBus class	13
2.9 The InfoBusMember interface	14
2.10 The InfoBusBean interface	15
2.11 The InfoBusMemberSupport class	16
2.12 The InfoBusBeanSupport class	18
3. Rendezvous	20
3.1 Events	20
3.2 Event listeners	20
3.3 Security considerations for rendezvous	22
3.4 Data item naming conventions	22
3.5 Data item name properties	23
3.6 DataFlavors and describing data items	23
3.7 The InfoBus class: listener management	24
3.8 The InfoBus class: firing events	24
3.9 The InfoBusEvent class	26
3.10 The InfoBusItemAvailableEvent class	26
3.11 The InfoBusItemRevokedEvent class	26
3.12 The InfoBusItemRequestedEvent class	27
3.13 The InfoBusEventListener interface	27
3.14 The InfoBusDataProducer interface	28
3.15 The InfoBusDataConsumer interface	28
3.16 The Proxy Listener classes	29
4. Data items	31
4.1 Overview	31
4.2 Example: A simple data item	31
4.3 Example: A spreadsheet data item	32
4.4 The DataItem interface	32
4.5 The DataItemView interface	33
4.6 Using the <code>awt.datatransfer.Transferable</code> interface	34
4.7 Synchronization and locking	34
4.8 Requirements for releasing data items	34
4.9 Access Interfaces	35
4.10 Mutability	36
4.11 The ImmediateAccess interface	36
4.12 The ArrayAccess interface	37
4.13 The ReshapeableArrayAccess interface	38
4.14 Using JDK Collection interfaces for InfoBus access	41

4.15 ArrayAccess and JDK Collections	42
4.16 Tree data items	42
4.17 Requirements for InfoBus-compliant data items	42
5. Database access interfaces	43
5.1 Overview: database access for the InfoBus	43
5.2 The RowsetAccess model compared to other access interfaces	44
5.3 DataBase Cursor and RowsetAccess design	44
5.4 The RowsetAccess interface	44
5.5 The ScrollableRowsetAccess interface	47
5.6 The DbAccess interface	48
5.7 The RowsetValidate interface	50
5.8 Releasing database resources	50
6. Monitoring changes to data items	51
6.1 The DataItemChangeManager interface	51
6.2 The DataItemChangeManagerSupport class	52
6.4 The DataItemShapeChangeListener interface	53
6.5 The DataItemChangeListenerSupport class	53
6.6 The DataItemChangeEvent class and event subclasses	54
6.7 Distribution and handling of DataItemChangeEvents	56
6.8 Examples of event propagation	58
6.9 The temporary InfoBusPropertyMap interface	59
7. Data controllers	60
7.1 Overview	60
7.2 Data controller examples	60
7.3 The DefaultController	60
7.4 InfoBus methods supporting data controllers	61
7.5 Composition of a data controller	61
7.6 Data controller priority levels	62
7.7 Maintaining producer and consumer lists in a data controller	62
7.8 Data controller event distribution	63
7.9 Example of event distribution process	64
8. Policy helper	66
8.1 The InfoBusPolicyHelper interface	66
8.2 The DefaultPolicy class	67
9. Exceptions	68
9.1 InfoBusMembershipException	68
9.2 DuplicateColumnException	68
9.3 ColumnNotFoundException	68
9.4 InvalidDataException	68
9.5 RowsetValidationException	68
9.6 UnsupportedOperationException	68
9.7 StaleInfoBusException	68
10. Guidelines for well-behaved InfoBus components	69
10.1 Membership	69
10.2 Data items	69
Appendix A: Changes in the InfoBus 1.2 Specification	70
Major Changes in the InfoBus 1.2 Specification	70
Minor Changes in the InfoBus 1.2 Specification	71
Appendix B - Deprecated Interfaces, Classes and Methods	72
B.1 DataItemChangeSupport replaced by DataItemChangeManagerSupport	72

Preface

Acknowledgments

This technology is the result of the work of several people in a few companies.

The InfoBus concept was created at Lotus by Douglass Wilson, who wrote the first draft of the spec and provided technical guidance.

The InfoBus development effort at Lotus is lead by Mark Colan, who is lead architect, and principal author and editor of the spec. Wendy Clarke was logistical lead. Chris Karle designed and wrote the specification for data controllers, policy helpers, and our security policies, gave lots of feedback on all aspects of the spec, and did most of the programming of the core classes and interfaces. Margaret O'Connell took over core and JCK development starting with InfoBus 1.2. Tom McGary designed and wrote the specification for the database access interfaces.

Andrew Peace assured the quality of the releases and write most of the test procedures. Jessica Goble wrote the JCK test suite for InfoBus 1.1 and helped qualify releases; she now serves as product management for InfoBus. Gwen Walton added new JCK tests for InfoBus 1.1.1 features. Lance McVay wrote an early draft of the spec and, along with Claudia Salzberg, gave editorial feedback. Dave Millen provided managerial support and plenty of free rein.

We would like to thank several key contributors at JavaSoft. Larry Cable proposed a better rendezvous mechanism which is part of the current design, suggested a naming convention, participated in partner reviews, and taught us all a lot about Java and beans. Krystyna Polomski helped us with the JCK test suite. Onno Kluyt is the logistical liason. Graham Hamilton, Gina Centoni, and Jeff Jackson have also provided valuable support.

Special thanks to Yuri Sharonin, Michael Riccio, Donald King, Mark Clark, and Larry Harris at (or formerly at) Oracle Corp. for extensive and detailed comments that led to several improvements, especially in the database access interfaces.

Finally, we thank the many people out there in the industry who read the spec and took the time to write detailed comments and suggestions. These ranged from reporting typos to suggesting an idea which led to a redesign of data items; all of these are helpful and welcome.

Target environments for this release

This specification describes a point release of InfoBus, called version 1.1.1. It adds to the first release (called InfoBus 1.1) a new interface for better Beans support, and four support classes. This release requires JDK 1.1.x or Java 2 Platform (see details in release notes); it does not use any Java 2 Platform features.

InfoBus release 2.0 is in development, and will require Java 2 Platform. Because it will make use of Java 2 Platform security features, it will not work with JDK 1.1.x.

InfoBus and its applications should work with any JVM that supports 100% Pure Java classes and has the correct version of the JDK environment. See the release notes for information on platforms we have specifically tested.

Questions and Feedback

Please send comments to infobus-comments@java.sun.com. To stay in touch with the InfoBus project, visit our web site at: <http://java.sun.com/beans/infobus>. Most comments and questions get a prompt answer when mailed to the infobus-comments address; harder questions take longer.

1. Overview

This specification is intended to provide standards by which a wide range of Java components acting as data producers and consumers can communicate data. It does this by defining a set of Java interfaces called the InfoBus interfaces. Java components that implement these interfaces are referred to as InfoBus components.

The InfoBus architecture facilitates the creation of applications built from Java Beans that exchange data asynchronously. This can be done by way of applets in an HTML page, or by way of beans assembled by a builder environment. InfoBus can also be used by arbitrary Java classes, including applets, servlets, and so on.

This first release of the InfoBus is called version 1.1 because it supports JDK 1.1. It can also be used with Java 2 Platform, though it does not use any Java 2 Platform features. The InfoBus 2.0 toolkit will exploit Java 2 Platform security and other features, and thus will not support JDK 1.1. Appendix A gives a preview of InfoBus 2.0 features.

1.1 Design of data flow between components

The InfoBus is designed for components working together in the same Java Virtual Machine (JVM). The current design does not directly deal with components working in separate JVMs, such as on different processors, though it does provide facilities that allow a bridge between JVMs to be built.

In general, all Java Beans loaded from a single class loader can “see” other beans from the same loader and make direct method calls on those beans. However, these cross-bean calls are currently based on well-known interfaces or base classes. Beans use “introspection” to “learn” or “discover” information about peer beans at run time. In such a case, one bean can infer an API supported by another by detecting certain “design patterns” in the names of methods discovered through introspection. By contrast, the InfoBus interfaces form a tightly typed contract between cooperating beans. No inferring is required, and procedure calls are direct.

The InfoBus interfaces allow the application designer to create data flows between cooperating beans. In contrast to an event/response model, where the semantics of an interaction depend upon understanding a bean-specific event and then responding to that event with bean-specific callbacks to the event raiser, the InfoBus interfaces have very few events and have an invariant set of method calls for all components. The semantics of the data flow are based on interpreting the contents of data that flows across the InfoBus interfaces, not in the names or parameters from events, nor in the names or parameters of callbacks.

1.2 Component types

Beans in an InfoBus application can be classified in three types: *data producers*, *data consumers*, and *data controllers*. An individual component can act as both data producer and data consumer. Between components, data flows in named objects known as “data items.” Data controllers are specialized components that mediate the rendezvous between producers and consumers.

1.3 Principal requirements for the InfoBus

The structure of an InfoBus application leads to two principal requirements for the InfoBus:

- The InfoBus should support the creation of interactive applications without requiring support of a “builder” application. That is, application designers should be able to assemble these applications using conventional web page editing tools. Further, these applications should run in standard HTML interpreted environments (browsers) without requiring specific extensions or support beyond the basic Java language environment. Note that this does not preclude enhanced capabilities in the presence of a JavaBeans-enabled environment.
- The InfoBus must support semantics that allow data to be communicated in a canonical format for consumption by multiple consumers. A canonical format involves both the encoding of data (numbers, strings, etc.) and navigation of data structure (rows, columns, tuples, etc.). Our intent is that mechanisms used to format and recover data be based as closely as possible on mechanisms already available from Java itself and JavaBeans.

InfoBus interfaces and properties should make sense in the context of JavaBeans-based builders. Builders should recognize the InfoBus properties, and be able to create useful wirings between components.

1.4 The InfoBus protocol for data exchange

Step 1. Membership — establishing InfoBus participation

Any Java component can connect to the InfoBus. This is done by implementing an InfoBusMember, obtaining an InfoBus instance, and having the member join it.

Step 2. Listening for InfoBus events

Once an object is a member of an InfoBus, it receives bus notifications by implementing an interface and registering it with the InfoBus. Two event listener interfaces are defined to support two basic types of InfoBus applications. A data consumer receives announcements about data availability by adding a consumer listener to the bus. Similarly, a data producer receives requests for data by adding a producer listener to the bus.

Step 3. Rendezvous on the data to be exchanged

In the InfoBus model, data producers announce the availability of new data as the data becomes ready (e.g., completion of a URL read, completion of a calculation, etc.). Consumers solicit data from producers as they require that data (applet initialization, button event, etc.). The rendezvous is by the name of the data. It is up to the application designer to designate the names for data items that can be exchanged.

Thus, all data producers and consumers must provide some mechanism for the application designer to specify data item names for rendezvous. For example, in a spreadsheet component, the user can “name” ranges on the sheet. This name is a natural mechanism for naming data that can be exported by the sheet in a role as a data producer. Similarly, a chart component needs a means of telling it what named data should be displayed in the chart.

Step 4. Navigation of structured data

Different data producers often use markedly different internal representations of data that is superficially similar. For example, a spreadsheet and a database both deal with tables, but store them quite differently. In a spreadsheet, the table of data might be represented as the output of a calculation (like matrix transpose), or as an array of formulas, whereas in a database the same information might be represented as the result of a join query.

A data consumer should not need a detailed understanding of the data producer’s internal data structures to use its data. A charting component should be able to draw a chart of a table from either a spreadsheet or a database whenever the table makes sense as a chart. In practice, this sort of information sharing requires consumer and producer to agree on a common encoding of the data. We have designed a set of interfaces for various standard protocols which are used to create data items with common access.

Step 5. Retrieval of an encoding of the data value

A data item can be retrieved as a String or a Java object. Java objects are typically object wrappers for primitive types such as Double or instances of other core classes such as Collection. The intent is to require as little specialized understanding of data formats on the part of the data consumer as possible.

Step 6. Optional: changing data

A consumer can attempt to change the value of data items. The producer enforces a policy on whether anyone can change data. With Java 2 Platform, it can also check for permissions from various consumers.

1.5 The nature of data items

To have useful communication between disparate data producers and consumers, some understanding of the content of the data streams is necessary. We need to establish what kinds of data are suitable for transport over the InfoBus. While the InfoBus protocols do not prohibit the exchange of very detailed data streams between components with understanding of each other’s semantics (e.g., a spreadsheet file import filter passing internal representations of the spreadsheet data to a sheet display component), it does not particularly facilitate this.

When deciding if data is suitable for transport, it is useful to ask if there is (1) more than one data consumer interested in this item, and (2) more than one data producer likely to produce an item of equivalent encoding. Essentially, does this item and its structure represent a class of data to be transferred? If yes is the answer, InfoBus is the framework.

2. Membership

2.1 InfoBus instances

The InfoBus class is at the heart of this technology. An instance of the InfoBus class is the meeting place where members can join.

A *named* InfoBus instance is one created on the basis of a name, which can be specified via a builder environment, externally-specified property, user input, or an application string. Any class that implements the InfoBusMember or InfoBusBean interfaces can join the InfoBus by name, even if they are not Applets.

A *default* InfoBus instance is one which is created on the basis of a generic name calculated from a DOCTYPE in a Component context. To find a default InfoBus the component must either be an Applet itself, or have an applet in its AWT containment hierarchy, so that we can traverse upwards to the applet in order to get the DOCTYPE. The caller itself need not be such a component, but it must be able to supply a Component context.

The advantage of using a default InfoBus is convenience. An Applet that joins its default InfoBus will be able to see other components on the same frame of the same web page that have joined their default InfoBus: because they have a common DOCTYPE, they share a common default InfoBus. This is convenient for Applets and other Java components that can supply a Component context.

Classes that do not have access to a Component context with a DOCTYPE (e. g., classes that are not part of an Applet) cannot join the Applet default InfoBus, but they can join an InfoBus by name.

The InfoBus class makes use of static methods and static data to manage and access a list of active InfoBus instances. In particular, a static method called InfoBus.get() is called by a prospective member to obtain an InfoBus instance for all future transactions. The get() method locates or creates an instance of the InfoBus class on the basis of the name or Component context, and manages it by way of a list of active instances.

Figure 2-1 shows the static parts of the InfoBus in a separate box, as if it were a different object, along with two InfoBus instances which members can join. It also shows a producer and a consumer which are about to join:

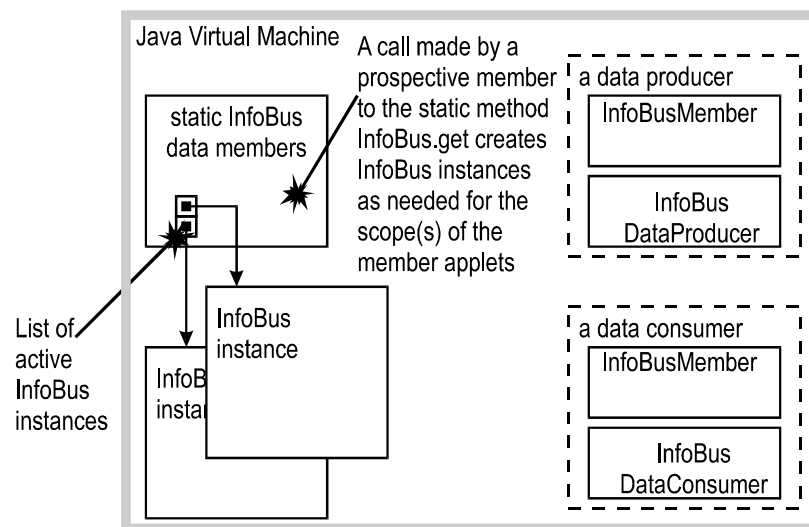


Figure 2-1 - InfoBus instance management before members have joined

2.2 Details of the membership process

A component must implement InfoBusMember to join the InfoBus, in preparation for receiving events about data on the bus. Membership is typically established during class initialization (for example, in applets, in the init() method). A

component can join more than one InfoBus, but it must use a separate InfoBusMember object for each InfoBus it wishes to join. A component that has created separate members to join more than one bus can use the same event listener object (described in the next chapter) to listen for events from more than one bus.

InfoBusMember can be implemented easily by creating an instance of InfoBusMemberSupport and delegating each method to the support class. The support class provides a method to join the bus called joinInfoBus(). This method starts a complicated sequence of calls between InfoBus and InfoBusMember to establish membership:

- joinInfoBus() calls get() to obtain an InfoBus instance to join, then calls InfoBus.join() to join it.
- join() checks the InfoBus instance to see if it is stale. If it is, StaleInfoBusException is thrown. The InfoBus instance can become stale if join() is erroneously called *after* the InfoBus instance has been released.
- join() calls the member's setInfoBus() method to set the member's InfoBus property to the InfoBus instance returned by get(). This can result in throwing PropertyVetoException, in which case the member was not allowed to join the bus returned by get().
- When exceptions are not thrown, the member's setInfoBus() implementation calls InfoBus.register(). register() adds the member to the list of members currently registered on this InfoBus instance, and calls the member's addInfoBusPropertyListener() to detect departures from the bus.
- When InfoBus.join() returns, joinInfoBus() finishes by calling release(), regardless of whether an exception was thrown.

Following a successful return from joinInfoBus(), a member can call methods on the InfoBus it joined by obtaining a reference to the bus by way of its own getInfoBus() method. This allows the member to add one or more event listeners and to fire events to other members, and is discussed in the next chapter.

Figure 2-2 shows the relationships between the InfoBus instance used for communications between a data producer applet and a data consumer applet after they have both joined the InfoBus:

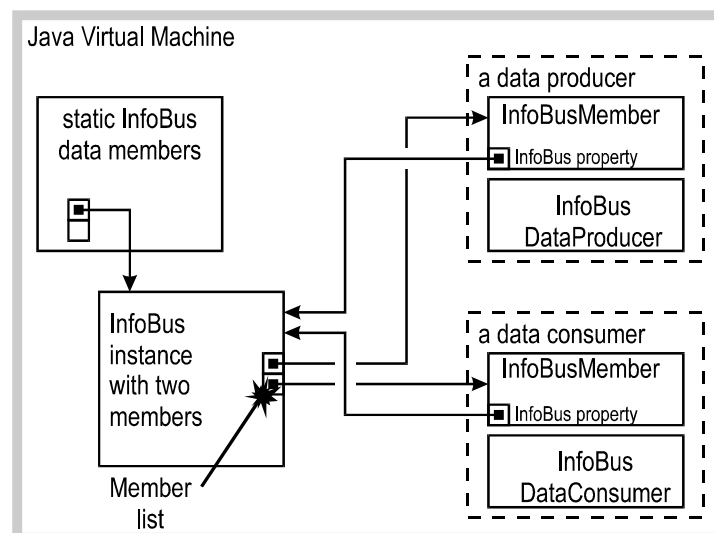


Figure 2-2 - A producer and consumer as InfoBus members

When the member calls the InfoBus.leave() method on the instance it previously joined, the InfoBus removes the member from its list, removes its property change listener from the member, and sets the member's InfoBus property to null. The close() method is called to check whether the InfoBus instance has lost its last members or event listener, and if so, releases the instance from the list of active InfoBus instances so that it can be garbage collected.

For maximum flexibility, an InfoBus component should support these three means of joining an InfoBus:

- If an InfoBus name is specified by way of a property or parameter, this name should be used when calling InfoBusMemberSupport.joinInfoBus().

- In the absence of a name to be used for an InfoBus, it should join the default InfoBus, if possible, supplying a Component context that can be used to locate a DOCBASE parameter.
- All beans should be prepared to accept a change to the “InfoBus” property on their InfoBusMember implementation, to support their use in a bean builder environment.

2.3 Revoking data items on changes to membership

When an InfoBus component is changing membership, leaving a bus with or without joining another, some questions arise about how it should deal with data items it produces or consumes. This specification allows such decisions to be made by the implementor, and we suggest that the behavior that is chosen be documented. In this section we mention some of the questions that arrive and some possibilities for answering them.

When a producer is leaving a particular InfoBus, it may choose to fire an InfoBusItemRevokedEvent on the InfoBus it is leaving, and a DataItemRevokedEvent on any outstanding data items still held by consumers, to indicate that it can no longer service methods called on such items. However, it may choose to continue servicing methods called on outstanding data items, even though it may not be on the same InfoBus as the consumers that are using the items, in which case it must not fire the revoked events. Upon joining the new bus, it may wish to reannounce its available items on the new bus.

Similarly, when a consumer is leaving a bus, it can choose to continue using a data item it has already obtained, or it can release it. It may wish to reannounce its requests for data items on the new bus.

2.4 InfoBus naming conventions

InfoBus instances can be named with any number of ‘unreserved’ characters as defined by the URI specification, except that they may not begin with ‘%’ or ‘-’. Names beginning with ‘%’ are reserved and must not be used. Illegal names may cause an exception to be thrown. See “Data item naming conventions” in the next chapter for more details.

2.5 Security considerations for membership

The InfoBus defines security policies in the InfoBusPolicyHelper interface, described in detail in Chapter 8. Some of these policies control whether a member is allowed to get, join, or register with an InfoBus instance. The DefaultPolicy class in InfoBus 1.1 implements this interface without any security checks, because this release supports JDK 1.1, which does not offer security support. InfoBus 2.0 will include a new implementation that provides security checks by using Java 2 Platform security features.

2.6 Monitoring member departures

A member ordinarily joins and leaves an InfoBus by way of the methods in InfoBusMember designed for these functions. In a browser environment, once a member joins a particular InfoBus, it generally remains as a member of the same bus until it terminates.

Many InfoBus participants will also be beans. When a bean is used in a builder environment, the builder may specify the InfoBus instance it wants the bean to join. Since beans will often join a default InfoBus as they initialize themselves, the builder may can reset the bean’s “InfoBus” property to put it onto a different bus.

An InfoBus instance that is losing a member needs to release itself after losing the last member. Since changing the InfoBus property on a member happens outside of calling the leave() method, the InfoBus implements PropertyChangeListener and adds itself as a listener for each of its members.

The InfoBus event listeners (InfoBusDataConsumer and InfoBusDataProducer) also need to know when their associated InfoBusMember is changing membership, so they also set property change listeners. When they are notified, they must remove themselves from the listener list for the old InfoBus instance, and (if the new property value is not null) add themselves to the new instance.

Figure 2-3 shows how the InfoBus and event listener objects use PropertyChangeListener objects as member data, and PropertyChangeSupport objects in the InfoBusMember implementation we provide via InfoBusMemberSupport:

A member object that has properties of its own, other than the “InfoBus” property, should override setInfoBus() to notify their own property change listeners, then call InfoBusMemberSupport.setInfoBus().

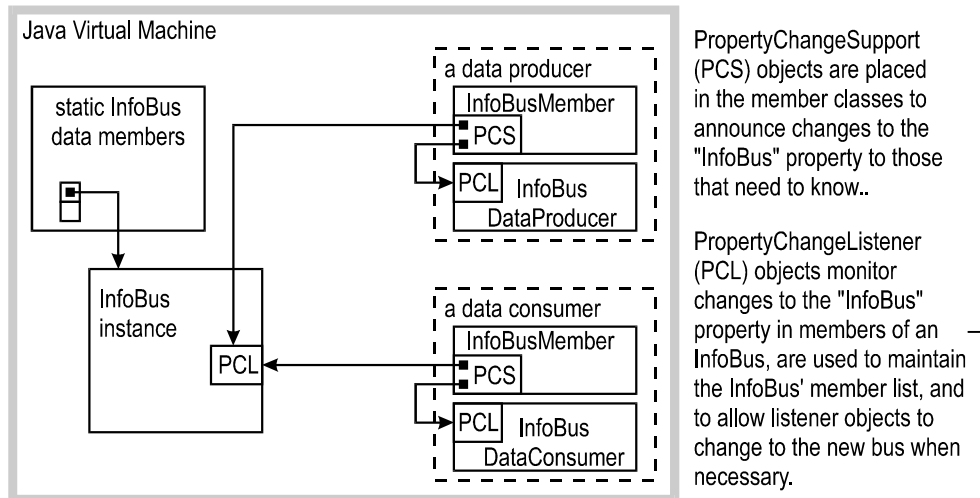


Figure 2-3 - InfoBus mechanism for monitoring member departures

2.7 About “Stale” InfoBus Instances

InfoBus instances are managed internally by keeping the active instances on a list. Whenever a particular InfoBus instance is losing a member, listener, or controller, the InfoBus checks to see whether there are any remaining members, listeners, or controllers, and if there are not, the InfoBus instance is removed from the active list. This allows the InfoBus instance to be reclaimed during garbage collection.

If an application has a reference to an InfoBus instance that has been removed from the active list, we refer to this instance as “stale.” Anyone who asks for an InfoBus instance that has the same name as this stale instance will get a different instance returned to them, because only the active instances can be found.

Methods called on a stale InfoBus instance (such as `join()`, `addDataProducer()`, `addDataConsumer()`, `addDataController()`, or `InfoBusMember.setInfoBus()`) will cause `StaleInfoBusException` to be thrown. When this `RuntimeException` is thrown, it generally indicates an error in the caller’s code. For example, consider this code snippet:

```
myMember.joinInfoBus("myBus");
InfoBus myBus = myMember.getInfoBus();
myMember.leaveInfoBus();
myBus.addDataConsumer(myConsumer);
```

Suppose also that the bus named “myBus” is not otherwise being used. Adding a consumer would throw an exception when called, because the bus was released when `leaveInfoBus()` was called, so `myBus` refers to a stale InfoBus instance.

A Bean container might have a similar bug by holding a reference to a stale bus, as in the following example:

```
InfoBus beanBus = InfoBus.get("beanBus");
bean1.setInfoBus(beanBus); // this works ok
beanBus.release();
// suppose bean1 leaves the bus on its own, then the following happens:
bean2.setInfoBus(beanBus); // this throws StaleInfoBusException
```

In this case, “beanBus” becomes stale when `bean1` leaves the bus, and `beanBus` refers to the stale bus. Setting the InfoBus on `bean2` to `beanBus` throws `StaleInfoBusException` because `beanBus` is stale.

The `get()` method adds an artificial reference to the bus it gets to ensure that the bus does not go stale before `join()` is called. In a multi-threading environment, it is possible that after successfully getting an InfoBus instance, another thread could cause leave the same bus, in which case the InfoBus would be cleaned up if nobody else was using it. The artificial reference is added to prevent it from being cleaned up before allowing the original thread to `join()` it.

The artificial reference must be released by calling the `release()` method immediately after joining the bus. Although the artificial reference is removed, the bus is not cleaned up (and does not become stale) when it still has at least one current member, producer listener, consumer listener, or data controller in place. If the `release()` method is not called, the `InfoBus` instance will not be reclaimed, even when it has no other members, listeners, or controllers associated with it.

In the Bean container example above, suppose a different thread is scheduled immediately after calling `InfoBus.get("beanBus")`, and this thread joins then leaves the same bus. Without the artificial reference, when the first thread tries to set `bean1`'s `InfoBus`, it would throw an exception. The artificial reference guarantees that `beanBus` does not become stale until it is released.

In summary, most `InfoBus` applications will never see a stale `InfoBus` if they use `InfoBusMemberSupport` methods `joinInfoBus()` and `leaveInfoBus()` for joining and leaving the bus, and instead of keeping a reference to the bus they joined, they call methods on their bus by getting the current property value, as in this example:

```
myMember.getInfoBus().addProducer(myProducer);
```

2.8 The `InfoBus` class

The `InfoBus` class must be used as we provide it (without subclassing) in the Java virtual machine. Because subclassing is not possible, the behaviors that a JVM may wish to override are collected into an interface called `InfoBusPolicyHelper`; we provide a default implementation of this interface in a class called `DefaultPolicy`.

The `InfoBus` plays a central role in membership, and the methods involved are described in this section. Most of the methods for membership are called by the `InfoBusMemberSupport` or `InfoBusBeanSupport` methods, not directly by applications.

The `InfoBus` class is also central to the rendezvous and event model used in the `InfoBus`, and these `InfoBus` methods are described in the next chapter.

All methods described in this section except for `getName()` are used by the `InfoBusMemberSupport` implementation class, and ordinarily are not called directly by most applications. Most `InfoBus` applications will join and leave the bus by way of `InfoBusMemberSupport` methods `joinInfoBus()` and `leaveInfoBus()`. A bean container may need to use the `get()` method when it wishes to force a contained bean to be a member of a particular `InfoBus`, in which case it must call `release()` after setting the `InfoBus` property on the contained beans.

```
public static synchronized InfoBus get(Component component)
```

```
public static synchronized InfoBus get(String busName)
```

These methods are used to get a reference to an `InfoBus` instance that can be used directly for joining, as for an ordinary `InfoBus` bean or applet, or indirectly to force membership on `InfoBus`-aware beans, as for a builder environment.

The first version specifies the *component* to be used for determining the name of the default `InfoBus` to be used for the Component's context. *component*, or one of its parent Components in the AWT containment hierarchy, must specify a `DOCBASE`, which is used to create a name according to rules established in `InfoBusPolicyHelper`. If no `DOCBASE` is found, this method returns `null`.

The second version uses a *busName* string as the name of the desired `InfoBus` to be specified. Named infobuses are useful for builder environments, classes that are not Components, and applications that wish to specify security constraints on particular buses by name. See section 2.3, "Infobus naming restrictions," for rules on naming `InfoBuses`. `IllegalArgumentException` may be thrown on an illegal name.

Whether the name is constructed by default or specified explicitly, both versions do their work by searching the existing `InfoBus` instances for a match and creating one if necessary. The reference to the found or created `InfoBus` is returned.

The `get()` method introduces an artificial reference to the `InfoBus` to ensure that the `InfoBus` instance is kept alive until a member joins it. This reference must be released by calling `release()` on the instance after completing the work (calls to `join()`, `setInfoBus()`, etc). Every `get()` must have a matching `release()`, regardless of whether the work in between succeeds or throws an exception. See also the example in the `join()` method for this class.

public synchronized void **join**(InfoBusMember *member*) throws PropertyVetoException

This method causes an InfoBusMember to join the specified InfoBus instance. It is generally used on an InfoBus instance returned by get(). For example:

```
InfoBus x = get("my InfoBus");    // get named InfoBus
x.join( myMember );              // join it
x.release();                     // release artificial reference
```

When this method is called on a stale InfoBus instance, the StaleInfoBusException is thrown.

We recommend that applications use the higher-level InfoBusMemberSupport.joinInfoBus() methods instead of calling InfoBus.join().

public synchronized void **release**()

This method is used for removing the artificial reference to an InfoBus instance set by calling get(). It should be called immediately after at least one member joins the bus obtained by get(). See also the example in the join() method.

When this method is called, it checks to see whether the InfoBus instance is no longer used, and allows it to be reclaimed thru garbage collection if the instance has no members, listeners, or artificial references. In the common case where it follows a join() call, the InfoBus instance is not garbage collected, because it has at least one member, i.e. the one that just joined.

InfoBus participants that use InfoBusMemberSupport.joinInfoBus() will typically not need to use this method. However, a Bean builder environment would use get() and release() as brackets around calls that set the InfoBus property on their contained Beans.

public String **getName**()

Returns a String with the name of the InfoBus. When the InfoBus was created by name, the name is returned. When the InfoBus took the default name for the DOCBASE, the name derived from DOCBASE is returned.

public void **register**(InfoBusMember *member*)

This method registers an InfoBusMember on the InfoBus's list of active members, and also causes the InfoBus to register itself as a PropertyChangeListener on the InfoBusMember's InfoBus property. It is called by InfoBusMemberSupport.setInfoBus(), and is not typically called directly by an InfoBus participant. When this method is called on a stale InfoBus instance, the StaleInfoBusException is thrown.

member is a reference to the InfoBusMember to add to the active member list.

public synchronized void **leave**(InfoBusMember *member*) throws PropertyVetoException

This method is called by implementations of InfoBusMember.leave when a *member* wishes to remove itself from the InfoBus it previously joined. We recommend that InfoBus applications use the InfoBusMemberSupport methods joinInfoBus() and leaveInfoBus() instead of InfoBus methods join() and leave().

public void **propertyChange**(PropertyChangeEvent *event*)

This method is called whenever an existing member has its "InfoBus" property changed by some means other than calling the leave() method, for example when a builder calls InfoBusMember.setInfoBus() to force it to talk to a different bus of its choice.

InfoBus applications do not call this method directly. It is called by methods in InfoBusMemberSupport when a member is leaving a given InfoBus instance.

2.9 The InfoBusMember interface

This interface must be implemented by classes that want to join an InfoBus. Using an interface to define the methods ensures that any container of an InfoBusMember can know how to cause a member to join an InfoBus.

The interface defines methods for joining and leaving an InfoBus, managing the "InfoBus" property via methods that conform to the Beans specification for a bound property of this name, and adding listeners for property changes on "InfoBus," as well as vetoable listeners.

To facilitate the implementation of this interface, we supply a class called `javax.infobus.InfoBusMemberSupport` which provides all required methods and member data and which can be used directly by using the class as a data member. We strongly recommend that this class be used for the implementation, rather than rolling your own.

```
public void setInfoBus(InfoBus newInfoBus) throws PropertyVetoException
```

This method is called by `InfoBus.join()` to set the member data reference to *newInfoBus* during the processing of `InfoBusMember.join()`. It can also be called by others that wish to force membership to a given `InfoBus`, such as by a builder tool that is arranging to have to applets talk over the same bus. Finally, it can be called with a `null` argument, such as in handling a call to `InfoBusMember.leave()`.

The `InfoBus` requires that an implementation of this method does the following:

- Broadcasts a `PropertyChangeEvent` to its `VetoableChangeListeners` and `PropertyChangeListeners`.
- Explicitly uses the `PropertyName` “InfoBus” in creating the event.
- Does not use `null` for the `PropertyName` field (which is allowed in the Java Beans spec, for example if multiple properties change).
- Sets the `InfoBusMember` as the Event source.

```
public InfoBus getInfoBus()
```

This method is an accessor for the current setting of the `InfoBus` property.

```
public void addInfoBusVetoableListener(VetoableChangeListener vcl)
```

```
public void removeInfoBusVetoableListener(VetoableChangeListener vcl)
```

These methods are called by a class that wishes to have a say about whether the “InfoBus” property for this class can be changed, to add or remove a vetoable listener object for the “InfoBus” property.

Listeners should allow the `InfoBus` property to be set to `null`, which indicates a class that is leaving the `InfoBus`, often because the class is shutting down.

```
public void addInfoBusPropertyListener(PropertyChangeListener pcl)
```

```
public void removeInfoBusPropertyListener(PropertyChangeListener pcl)
```

These methods are called by a class that wishes to be notified when the “InfoBus” property for this class is about to change. The methods add or remove a listener object to enable notification.

2.10 The `InfoBusBean` interface

This interface extends `InfoBusMember` to add support for a new property called *infoBusName* property. This property allows `InfoBus` membership to be configured in an Bean builder environment, even when the builder is not `InfoBus`-aware, by setting it to a `String` using the property editor.

Changing this property to a new name has the side effect of attempting to leave any bus the member has previously joined, and then joining a bus with the new name. Setting the *infoBusName* property to an empty string causes the member to leave the current bus and not attempt to join another. When the new name is set to “-default”, we recommend that components that are also Applets respond by joining the Applet default `InfoBus`.

Changing the *infoBusName* also causes the *InfoBus* property to change, so that it refers to one that has the indicated name. The latter property may have listeners to monitor or prevent changes, in which case the listeners effectively monitor or prevent changes to both properties. In the event that a listener on the *InfoBus* property vetoes the change, the *infoBusName* property must not change, and an exception is thrown.

Most of the time, the *infoBusName* and *InfoBus* properties track each other’s values, so that when one is `null`, the other is also `null`, and when one is non-`null`, the other is also non-`null`. This occurs because setting the name affects membership, and methods affecting membership usually change the name. However, there are times when the *infoBusName* property can be non-`null` even though the *InfoBus* property is `null`. One case is when the two-argument constructor in `InfoBusBeanSupport` is called to specify an initial value for the property; this constructor does not cause membership to occur. The other is when the *infoBusName* property has been set in a builder, then the Bean is serialized. When such a Bean is deserialized, *infoBusName* has a non-`null` value,

yet the Bean is not a member of the named bus, so *infoBusName* is `null`.

Just as the *InfoBusMember* interface is accompanied by a full implementation class, allowing the programmer to delegate all method calls to the corresponding methods in *InfoBusMemberSupport*, the *InfoBusBean* interface is accompanied by an implementation class, allowing the programmer to delegate method calls to the corresponding methods in *InfoBusBeanSupport*.

Synchronization requirements

The implementations of *setInfoBus()*, *getInfoBus()*, *setInfoBusName()*, and *getInfoBusName()* must be synchronized on the same Object to prevent more than one of these from running at a time in different threads.

Persistence requirements

The *InfoBus* property should be treated as transient. When the *InfoBusBean* implementation is *Serializable*, which we recommend, the variable that manages the setting of the *InfoBus* property should be declared as transient. In addition, the *BeanInfo* for the Bean should return a *PropertyDescriptor* for the *InfoBus* property, with these two attributes:

- *PropertyDescriptor.setHidden(true)* should be called to mark this property as hidden.
- *PropertyDescriptor.setValue("transient", "true")* should be called to mark this property as transient.

The *infoBusName* property should have persistent state, and should be configurable in a property sheet. Therefore it should *not* be hidden, and the transient value can be omitted or explicitly set to false.

`public void setInfoBusName(String newBusName) throws InfoBusMembershipException`

This method must set the *infoBusName* property to *newBusName*, and as a side effect, cause a change of membership to the indicated bus.

newBusName specifies the name of a new bus to join, as defined in Section 2.3, *InfoBus* naming conventions. *newBusName* can be an empty String or `null` reference (the two must be treated the same) to indicate that the old membership should be terminated and no new bus should be joined. *newBusName* can also be “-default”, in which case this method should attempt to join the Applet default *InfoBus*, if possible, and leave the *infoBusName* property set to “-default”. The *infoBusName* property must be maintained as persistent member data.

Unless a vetoable listener prevents a change to membership, setting this property must terminate any prior membership and, if *newBusName* is not an empty string or `null` reference, join a bus obtained with *newBusName* (or the Applet default *InfoBus* if it is “-default”). If *newBusName* is `null` or an empty String, *setInfoBus(null)* must be called to terminate the current membership without joining a new bus.

When a vetoable listener on the *InfoBus* property throws *PropertyVetoException* to prevent a change in membership, *setInfoBusName()* must throw *InfoBusMembershipException*, and the value of the *infoBusName* property and the existing bus membership must remain unchanged.

`public String getInfoBusName()`

This method returns a String holding the current value of the *infoBusName* property, or `null` if there is no name.

In most cases, the String returned from this method is the same as the one returned by calling *getInfoBus().getName()*. However, if the *infoBusName* property has been set to “-default”, *getInfoBusName()* returns “-default”, whereas *getInfoBus().getName()* returns the actual, resolved name of the Applet default bus. If the member has not joined a bus, or has left a bus, *getInfoBusName()* returns `null` and *getInfoBus().getName()* is invalid (because *getInfoBus()* is `null`).

`public void setInfoBus(InfoBus newInfoBus) throws PropertyVetoException`

This method must change the *InfoBus* property, as detailed in *InfoBusMember.setInfoBus()*, but it must also change the value of the *infoBusName* property, unless a vetoable listener prevents a change. The latter property is set to the name of the *newInfoBus*, or to an empty String if *newInfoBus* is `null`.

2.11 The *InfoBusMemberSupport* class

This class provides code that can be used for implementing the *InfoBusMember* interface. Classes that implement

the `InfoBusMember` interface can create an instance of this implementation class as member data, and expose the `InfoBusMemberSupport` methods to the outer class by creating a wrapper method. Some methods in this class (`joinInfoBus()`, `leaveInfoBus()`) are not required by the interface, but are handy for users of the class. In the following example, we show the wrapper for `setInfoBus()`. Other wrappers are similar to this approach:

```
class myMember implements InfoBusMember
{
    private InfoBusMemberSupport m_memberSupport = new InfoBusMemberSupport();
    public void setInfoBus(String name) throws PropertyVetoException
    {
        m_memberSupport.setInfoBus(name);
    }
    // other wrapper methods go here
}
```

Constructor

```
public InfoBusMemberSupport(InfoBusMember member)
```

The constructor sets the `InfoBus` reference `member` to `null`, and creates an instance of each of the `VetoableChangeSupport` and `PropertyChangeSupport` objects.

The *member* parameter is a reference to the `InfoBusMember` instance that contains this `InfoBusMemberSupport`, and is used for property change notifications on the “InfoBus” property.

Membership methods

```
public synchronized void joinInfoBus(String busName)
    throws InfoBusMembershipException, PropertyVetoException
public synchronized void joinInfoBus(java.awt.Component component)
    throws InfoBusMembershipException, PropertyVetoException
```

These methods are the preferred way for an `InfoBus` component to join an `InfoBus`. Following a successful return from these methods, the `InfoBus` property has a reference to the bus it joined, which can be used for adding event listeners, firing events, or making other calls on the `InfoBus` instance it has joined.

If either method is called when the member has previously joined a bus but has not left it, `InfoBusMembershipException` is thrown, and the membership is unchanged.

The first method allows a component to join an `InfoBus` by a *busName* it specifies as an argument. Legal names are described in section 2.3, `InfoBus` naming restrictions. This method can be used by any member to join a bus, whether the application is an Applet or not.

The second method is convenient for classes that either have a `Component` with a `DOCBASE` or have a parent in the AWT hierarchy that has a `DOCBASE`. An Applet is a common example of such Components. It allows these classes to join the “Applet default `InfoBus`,” which is simply a bus whose name is calculated from the `DOCBASE`. Applets with the same `DOCBASE` can expect to see each other’s events after joining with this method.

```
public synchronized void leaveInfoBus()
    throws InfoBusMembershipException, PropertyVetoException
```

This method is called by an application after removing its event listeners when it is finished with a given `InfoBus`. It must be called before the application shuts down or before joining a different bus.

Methods to manage the “InfoBus” property

```
public synchronized void setInfoBus(InfoBus newInfoBus) throws PropertyVetoException
```

This method is called to set the `InfoBus` property for a given member. Setting this property results in changes to membership: any prior membership is terminated, and if *newInfoBus* is not `null`, the member joins that bus. Any

vetoable or property change listeners are notified about the change according to the standard rules.

This method is typically called by a container application, such as a Bean builder environment, to cause InfoBus members it contains to be members of a particular bus.

PropertyVetoException is thrown when a VetoableChangeListener on the member refuses to allow the change in membership. StaleInfoBusException is thrown when *newInfoBus* refers to InfoBus instance which is stale.

The implementation in this class does the following:

```
public InfoBus getInfoBus()
```

The implementation in this class returns the current value of the “InfoBus” property.

```
public void addInfoBusVetoableListener(VetoableChangeListener vcl)
```

```
public void removeInfoBusVetoableListener(VetoableChangeListener vcl)
```

These methods call addVetoableChangeListener() or removeVetoableChangeListener() on a VetoableChangeSupport object in member data.

```
public void addInfoBusPropertyListener(PropertyChangeListener pcl)
```

```
public void removeInfoBusPropertyListener(PropertyChangeListener pcl)
```

These methods call addPropertyChangeListener or removePropertyChangeListener on a PropertyChangeSupport object in member data.

2.12 The InfoBusBeanSupport class

```
public class InfoBusBeanSupport extends InfoBusMemberSupport
    implements InfoBusBean, java.io.Serializable
```

This class provides code that is recommended for implementing the InfoBusBean interface. It implements InfoBusBean in compliance with all requirements specified for that interface.

InfoBusBeanSupport implements the Serializable interface to provide persistence for the setting of the *infoBusName* property. The *InfoBus* property in this class is declared as transient. Users of this class should also provide PropertyDescriptors for these two properties according to the requirements described for the InfoBusBean interface.

The get and set methods for the *InfoBus* and *infoBusName* properties are synchronized to prevent more than one of these methods from running at the same time, as is required for the InfoBusBean interface.

Classes that implement the InfoBusBean interface can create an instance of this implementation class as member data, and expose the InfoBusBeanSupport methods to the outer class by creating a wrapper method.

```
public InfoBusBeanSupport(InfoBusMember parent)
```

The constructor initializes an InfoBusBeanSupport object. The *parent* parameter, usually a reference to an InfoBusBean that created the instance of InfoBusBeanSupport, is used as the source of all PropertyChangeEvents. If *member* is null, this InfoBusBeanSupport object is specified as the source of PropertyChangeEvents.

If *parent* is an instance of Component, calling setInfoBusName(“-default”) will attempt to use this Component to locate the DOCBASE parameter for resolving the name of the Applet default InfoBus.

```
public InfoBusBeanSupport(InfoBusBean parent, String initialBusName)
```

This constructor calls the other constructor, then sets the initial value of the infoBusName property to the value specified by initialBusName. Although the property has an initial value, this constructor does not cause membership to occur. Use of this constructor variant allows a Bean to have a preset default value for the *infoBusName* property other than null.

```
public void setInfoBusName(String newBusName) throws InfoBusMembershipException
```

This method is a complete implementation of InfoBusBean.setInfoBusName() as described in Section 2.9, The InfoBusBean interface.

When *newBusName* is specified as “-default”, this method attempts to use the *parent* of this instance (see constructor) as a Component for resolving the name of the Applet default InfoBus. For applications that can provide such a Component, but it is different from the constructor’s *parent* parameter, we recommend that the implementation of InfoBusBean.setInfoBusName() first compare *newBusName* to “-default”, and if they match, join the Applet default InfoBus by calling joinInfoBus(Component) instead of delegating to this implementation.

```
public String getInfoBusName()
```

This method returns a String holding the current value of the infoBusName property, or an empty String if there is no current name set.

```
public void setInfoBus(InfoBus newInfoBus) throws PropertyVetoException
```

This method overrides that of InfoBusMemberSupport to change to the indicated bus, but it also changes the value of the *infoBusName* property. The property is set to the name of the *newInfoBus*, or to null if *newInfoBus* is a null reference.

Except for the addition of changing the *infoBusName* property, the method functions identically to InfoBusMemberSupport.setInfoBus().

```
public void rejoinInfoBus() throws InfoBusMembershipException
```

This method can be called after deserializing a member to cause the member to join a bus whose name was specified before the member was serialized. It does nothing if called when the member has already joined a bus (*InfoBus* property is not null), or if the *infoBusName* property is an empty String.

InfoBusMembershipException is thrown when a VetoableChangeListener refuses to permit the *InfoBus* property to be changed.

3. Rendezvous

This chapter describes the event-based mechanism used by InfoBus components to announce data availability and request data among other components on the bus. We refer to the negotiation for data as the ‘rendezvous.’

3.1 Events

Events are sent by the InfoBus to listeners for each component on the bus. Three types of events are defined:

- InfoBusItemAvailableEvent - an event which is broadcast on behalf of a producer to let potential consumers know about the availability of a new data item through the InfoBus.
- InfoBusItemRevokedEvent - an event which is broadcast on behalf of a producer to let consumers know that a previously available data item is no longer available.
- InfoBusItemRequestedEvent - an event which is broadcast on behalf of a consumer to let producers know about the need for a particular data item that they may be able to supply.

The three InfoBus events are subclasses of a common base class, each with methods needed for their particular task.

The InfoBus class provides methods that create and broadcast these event objects on behalf of producers and consumers, including `fireItemAvailable()` and `fireItemRevoked()` for use by producers, and `findDataItem()` and `findMultipleDataItems()` for use by consumers.

3.2 Event listeners

Once a class has joined an InfoBus, it needs to provide an event listener to the InfoBus in order to receive events from the InfoBus. InfoBus components listen for events to discover the availability or revocation of data items, or to hear requests for data items, or both. InfoBus defines interfaces `InfoBusDataProducer` and `InfoBusDataConsumer` which extend `InfoBusEventListener` to indicate whether a component is a data producer, a data consumer, or both. The API details for `InfoBusEventListener`, `InfoBusDataProducer`, and `InfoBusDataConsumer` are discussed later in this chapter.

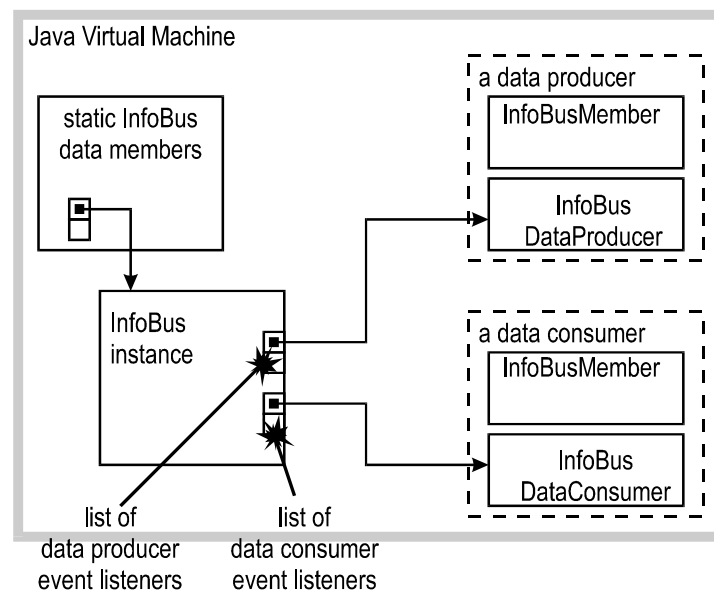


Figure 3-1 - A producer and a consumer on the bus, ready to receive events

Data producers

A data producer is an InfoBus participant that implements `InfoBusDataProducer` to listen for requests and announces data availability or revocation by firing events on the bus.

The producer calls `addDataProducer()` to begin receiving events. Applets typically do this during the execution of their `start()` method so they begin to receive events when the page is activated, and call `removeDataProducer` during the execution of the `stop()` method. Following this protocol reduces overhead when the page is not active. (However, with some browsers it may be possible to use an instance of the InfoBus for communication between applications on different web pages. The browser must not “prune” Java applets as pages are changed for this to work, and the InfoBus applications must not remove their listener on `stop()` in order to receive or send InfoBus events.)

Producer events are generated by calling methods `fireItemAvailable()` and `fireItemRevoked()` on the InfoBus class, which send these events to registered consumers. The producer handles request events via `dataItemRequested()`. If the producer can provide the requested data, it stores a reference to the data item by calling `setDataItem()` on the event, otherwise it simply returns.

Data consumers

A data consumer is an InfoBus participant that implements `InfoBusDataConsumer` to listen for availability and revocation events, and requests data items by firing events on the bus. Similar to the producer mechanism, it controls events by calling `addDataConsumer()` and `removeDataConsumer()`.

The consumer finds out about new data by handling `dataItemAvailable()` and revocation of announced data by handling `dataItemRevoked()`. It decides whether it is interested in the data by inspecting the data item name or data flavors obtained from the event via `getDataItemName()` or `getDataFlavors()`. If it wants the data, it can obtain it directly from the producer by calling `requestDataItem()` on the event.

The consumer can request a data item by name even if it has not been announced by a producer. For example, `findDataItem()` can be called to find a data item by name. Such blind requests are often required when a consumer initializes, in case the producer announced the data item before the consumer had completed initialization. If the data is unavailable from any producer, `null` is returned to the caller.

If more than one registered producer is capable of producing the data, the first one that supplies the data item satisfies the request. A consumer can call `findMultipleDataItems()` to get data items from all registered producers that are capable of supplying the data, and choose for itself which one it will take.

A component can be both a producer and a consumer by implementing `InfoBusDataProducer` and `InfoBusDataConsumer`. An applet might do this to filter data or translate it to a new form for a consumer.

Producers and consumers cannot directly create and broadcast events to others on the bus, because the constructors for events are not public, and because producers and consumers do not have access to the list of other producers and consumers. The InfoBus design intentionally prevents the use of custom events, since event traffic on the bus limits scalability.

Data controllers do have the ability to send events to producers and consumers they choose, to allow them to mediate the flow of events among other components. However, data controllers use a different mechanism for handling events from consumers and producers, and are therefore not event listeners themselves.

Figure 3-1, above, shows a producer and a consumer just after they have provided event listeners to the InfoBus instance they belong to. The InfoBus instance has a list of consumer listeners and a list of producer listeners that is separate from other InfoBus instances, in order to control the scope of conversations between applets and to reduce traffic. Although the membership connections and change listeners are not shown in Figure 3-1 in the interest of keeping the diagram simple, they would still be in place.

Provision for multiple event listeners

Figure 3-1 shows applications that have only one `InfoBusEventListener` associated with each `InfoBusMember`. The reason we did not combine these into one interface is that it will often be convenient for an application to have more than one event listener, each specialized to a particular data item of interest. InfoBus components can register as many event listeners as they need.

For example, consider a shared technology that supports the notion of the currently active selection by way of a data item called “CurrentSelection.” The provider of this item is likely to be in a different class than, for example, the provider of a collection of cells, and the use of multiple event listeners makes structuring the classes more convenient.

3.3 Security considerations for rendezvous

Security during the rendezvous process can be approached from two granularities: security checks before permitting distribution of an `InfoBusEvent` constitute a large-grained approach, while security checks upon delivery of an `InfoBusEvent` to a producer or consumer constitute a fine-grained approach.

The fine-grained security approach occurs in the producers and consumers themselves. An example using Java 2 Platform mechanisms is a producer who creates a data access permission akin to the `FilePermission` class, with system security policy files that enumerate classes that have that access permission. When this producer receives an `InfoBusItemRequested` event, it can call the `AccessController`'s `checkPermission` method to verify that all objects in the call stack — which will include the requesting consumer — have the necessary access permission before releasing data.

Consumers that wish to implement this kind of fine-grained permission checking need to take the additional step of implementing the `javax.infobus.SecureConsumer` interface (forthcoming; see Appendix A). Without the `SecureConsumer` interface, the data that a consumer requests is returned by the `InfoBus`, and the producer providing that data has no presence in the call stack. By implementing `SecureConsumer`, each producer that returns data is actually calling the `SecureConsumer.setDataItem()` method, allowing the `SecureConsumer` to perform an `AccessController.checkPermission()` before accepting and processing the data.

The `InfoBusPolicyHelper` provides the methods necessary to implement the large-grained approach: for each of the three event types, there is a matching `InfoBusPolicyHelper` call that is made before distributing the requested event. The implementer of `InfoBusPolicyHelper` can use security mechanisms such as those in Java 2 Platform to determine if the calling participant is permitted to generate the `InfoBusEvent`. Chapter 8 describes the `InfoBusPolicyHelper` in more detail.

The current, 1.1-compatible version of the `DefaultController` performs none of the large-grained checks in the rendezvous methods. Although the next version of the `DefaultController` will use 1.2 security checks on other `InfoBus` activity, it will also not implement the large-grained security checks before distributing events. Performing the rendezvous checks by default produces unwanted overhead in code execution as well as overhead in management of system security policies. The `InfoBusPolicyHelper` checks done during membership processing, combined with fine-grained checks done by individual consumers and producers during rendezvous, are a sufficient and optimal means of creating a secure `InfoBus` application. Systems that wish the additional layer of security described here as large-grained rendezvous checks have the means of implementing it by providing a custom policy helper.

An `InfoBus` participant should create classes that implement `InfoBusDataProducer` and `InfoBusDataConsumer` separately from one that defines other methods or implements other interfaces, especially `InfoBusMember`. We suggest this separation because the listener interfaces are available from events and data items, and introspection allows access to other methods available on these objects. In particular, if `InfoBusMember` is in the same class, it would allow access to `setInfoBus()`, which a malicious application could use to force a member onto a different bus.

3.4 Data item naming conventions

Data items can be announced as available by name, and consumers can request data items by name. Data items can be named using the recommended naming conventions described in this section, based on Universal Resource Identifiers (see <http://ds.internic.net/rfc/rfc1738.txt>). However, data items are not required to follow these conventions. The only requirements for data item names are:

- No data item name can begin with the `'%'` character, which defines a reserved space for data item names.
- If the data item name starts with `"infobus:"` or `"/"`, the same as a URI does, it must follow all of the rules for the convention.

URI-based InfoBus naming convention

```
<infobus_uri>          ::= <abs_infobus_uri> | <rel_infobus_uri>
<abs_infobus_uri>      ::= <infobus_scheme> ':' <rel_infobus_uri>
<infobus_scheme>       ::= 'infobus'
```

```

<rel_infobus_uri>      ::= '/' <infobus_name> { <producer_id>
                               '/' <infobus_data_item_name>

<infobus_name>         ::= <unreserved>*
<producer_id>          ::= '/' <producer_class> { '/' <producer_discriminator> }*
<producer_class>       ::= fully-qualified Java class name (with dot separators)
<producer_discriminator> ::= <unreserved>*
<infobus_data_item_name> ::= <unreserved>*
<unreserved>          = ALPHA | DIGIT | safe | extra
<extra>                = '!' | '\'' | '(' | ')' | ',' |
<safe>                 = '$' | '-' | '_' | '.'

```

These notes apply to item names, though they are not suggested by the BNF description above:

- The `infobus_name` is the one specified for a named InfoBus, or when using a default name for a DOCTYPE, can be obtained by calling `myMember.getInfoBus().getName()`. Note that `infobus_name` cannot begin with '-', and the use of '%' as the leading character is reserved. (see Chapter 2).
- The `producer_class` is a qualified Java class name with dot separators, like `com.lotus.esuite.sheet`.

3.5 Data item name properties

Many producers and consumers support only one data item at a time, and the rendezvous takes place when a name for the item matches the name in an InfoBusEvent. For such components, we recommend that the name of the item they consume or produce be externally configurable by supporting a property and/or Applet parameter called *inputItemName* or *outputItemName*, respectively. Following this convention will make it easier for developers to configure InfoBus-aware components.

Components that consume or produce more than one data item at a time, or do something other than a simple name match for the rendezvous, may expose names for the data items as makes sense to these components.

The property is intended to be preconfigured or set during initialization. However, if the value of a property with a data item name changes after the item has been announced, we recommend that the application revoke the item and send a new available event with the new name.

3.6 DataFlavors and describing data items

DataFlavors and the MIME type strings they expose can be used to describe data items provided in the InfoBus rendezvous. This is intended as a hint to the consumer, to determine before requesting data from an available event whether it can make use of it. It is also a hint to the producer, to determine whether it can supply the data in response to a request event in a useful form.

The DataFlavors can be exposed in available and request events with MIME strings. The InfoBus class defines static constant Strings that should be used for this purpose. The strings are in the following form:

```
application/x-java-infobus;class=<fully-qualified-interface-name>
```

where *<fully-qualified-interface-name>* is one of the following:

- the package qualifier plus the name of one of the access interfaces defined in this specification (ImmediateAccess, ArrayAccess, RowsetAccess, ScrollableRowsetAccess, and DbAccess). Example: MIME_TYPE_IMMEDIATE_ACCESS is defined as "application/x-java-infobus;class=javax.infobus.ImmediateAccess"
- the package qualifier plus the name of one of the four access interfaces defined in JDK Collections (Collection, Map, Set, and List). Example: MIME_TYPE_COLLECTION is defined as "application/x-java-infobus;class=java.util.Collection"
- the package qualifier plus the name of the DataItem class, which is used to indicate that if none of the preceding DataFlavors are available, that any other access type should be provided. Example: MIME_TYPE_ANY_ACCESS is defined as "application/x-java-infobus;class=javax.infobus.DataItem".

In announcing the availability of a data item, a producer can supply an array of `DataFlavors` available for the item by way of a parameter on the `InfoBus.fireItemAvailable()` method. The value of this parameter, whether a reference to an array or just `null`, is available to the consumer by way of `InfoBusItemAvailableEvent.getDataFlavors()`. The flavors are the union of the three groups listed above. See also Chapter 4 for more discussion on data flavors.

Similarly, a consumer can indicate its preferred format for a data item by providing an array of `DataFlavors`, ordered by preference, on the `InfoBus.findDataItem()` and `findMultipleDataItems()` methods.. The value of this parameter, whether a reference to an array or just `null`, is available to the producer by way of `InfoBusItemRequestedEvent.getDataFlavors()`. Because this is a hint to the producer, and the producer may supply an item that is not one of the preferred flavors, the consumer must code defensively to ensure that it has an item of a type it can use, for example by using the `instanceof` operator.

3.7 The InfoBus class: listener management

The `InfoBus` class is involved with membership and with rendezvous. The membership methods are discussed in detail in the previous chapter. This section discusses the `InfoBus` methods used to manage event listeners.

```
public synchronized void addDataProducer (InfoBusDataProducer producer)
public synchronized void removeDataProducer (InfoBusDataProducer producer)
public synchronized void addDataConsumer (InfoBusDataConsumer consumer)
public synchronized void removeDataConsumer (InfoBusDataConsumer consumer)
```

These methods add or remove event listeners from the list of data producers or data consumers maintained by each `InfoBus` instance.

The add methods should be called after the component has joined a bus (see Chapter 2). After adding an event listener, the class will begin to receive requests from data consumers on the same bus. If the add methods are called on a stale `InfoBus` instance, the `StaleInfoBusException` is thrown.

The remove methods must be called to remove all listeners before shutting down the application to release references from the `InfoBus` to the application class and to allow the `InfoBus` instance to be released.

3.8 The InfoBus class: firing events

This section discusses `InfoBus` methods used by producers, consumers, and data controllers for firing events, and is organized by the type of event fired. The first method in each group is one that is used by a producer or consumer to fire an event. This method actually defers the distribution of events to registered data controllers, or the default controller if no data controllers are registered; data controllers must not call this method.

Each event group also has methods to fire events to a specific target or a `Vector` of targets. Events fired with these methods are delivered directly to the indicated targets, not to other controllers. These methods can be called by data controllers, and must not be called by data producers; see Chapter 7 for more information.

Any method that fires events (including the ‘find’ methods) can throw `java.security.AccessControlException`. Because this is a runtime exception, the use of ‘try...catch’ clauses is optional.

Synchronization requirements for firing events

The `InfoBus` unites multiple components that work together as one application. Each `InfoBus` component must be aware that their code may execute in a multithreaded fashion, even when they do not spawn threads themselves, since they may be called from other components that do use multiple threads.

The `InfoBus` requires that when an available event for a particular data item name and producer is fired, it must be received by all listeners before the corresponding revoked event (i.e., a revoked event from the same producer with the same data item name) is fired. In order to guarantee that this requirement is met, before firing the available event, the producer must temporarily disable its ability to send the corresponding revoked event by using synchronization techniques appropriate for multithreaded operation.

This can be accomplished using a synchronization block around code that fires available and revoked events. In implementing this, a component must not specify its `InfoBus` as the parameter to the Java `synchronized` keyword,

as this can cause a deadlock to occur.

Firing the item available event

```
public void fireItemAvailable(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataProducer producer)
```

This method is called by producers to create an instance of `InfoBusItemAvailableEvent` and send it to data consumers on the bus, indicating the *dataItemName* and its *producer*. Producers can specify the *flavors* of data they can supply, or use `null` if they don't care to describe the data. Consumers can examine the offered *flavors* and decide whether they can use the data, or decide after requesting the data. Data controllers must not call this method.

```
public void fireItemAvailable(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataProducer source, InfoBusDataConsumer target)
```

```
public void fireItemAvailable(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataProducer source, Vector targets)
```

These methods are designed for use by data controllers. The first method creates an `InfoBusItemAvailableEvent` and delivers it to *target*. The second method creates a single `InfoBusItemAvailableEvent` and delivers it to all consumers specified in the `Vector targets`. All elements specified in *targets* must implement `InfoBusDataConsumer`. The *targets* `Vector` is copied by the `InfoBus` before distribution begins.

Firing the item revoked event

```
public void fireItemRevoked(String dataItemName, InfoBusDataProducer producer)
```

This method is called by producers to create an instance of `InfoBusItemRevokedEvent` and send it to data consumers on the bus, indicating the *dataItemName* and its *producer*. Data controllers must not call this method. Producers should call this method when a data item that has been announced as available will no longer be available.

```
public void fireItemRevoked(String dataItemName,  
    InfoBusDataProducer source, InfoBusDataConsumer target)
```

```
public void fireItemRevoked(String dataItemName,  
    InfoBusDataProducer source, Vector targets)
```

These methods are designed for use by data controllers. The first method creates an `InfoBusItemRevokedEvent` and delivers it to *target*. The second method creates a single `InfoBusItemRevokedEvent` and delivers it to all consumers specified in the `Vector targets`. All elements specified in *targets* must implement `InfoBusDataConsumer`. The *targets* `Vector` is copied by the `InfoBus` before distribution begins.

Firing the item requested event (find methods)

```
public Object findDataItem(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataConsumer consumer)
```

This method is called by consumers to create an instance of `InfoBusItemRequestedEvent` and send it to data producers on the bus, indicating the *dataItemName* and *consumer* that wants it.

The consumer can specify its preferred *flavors* or just say `null`; see “DataFlavors and describing data items” in this chapter for more information.

The event is sent to each registered producer until a producer supplies a data item, at which time the data item is returned to the caller. The order of polling the data producers for such requests is indeterminate. If no producer satisfies the request, the method returns `null` to indicate that the requested data item is not available. Because data controllers control the distribution of this type of event, they must not call this method.

```
public Object findDataItem(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataConsumer consumer, InfoBusDataProducer target)
```

```
public Object findDataItem(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataConsumer consumer, Vector targets)
```

These methods are designed for use by data controllers. The first method creates an `InfoBusItemRequestedEvent`

and delivers it to *target*, then returns a response to the request or `null` if *target* does not fill the request.

The second method creates a single `InfoBusItemRequestedEvent` and delivers it to the producers found in the *targets* Vector until one producer responds by filling the request or all producers have been queried. The method returns the response object if a producer filled the request, or `null` if no producer responded. All elements specified in *targets* must implement `InfoBusDataProducer`. The *targets* Vector is copied by the `InfoBus` before distribution begins.

```
public Object[] findMultipleDataItems(String dataItemName, DataFlavor[] flavors,  
                                       InfoBusDataConsumer consumer)
```

This method creates an instance of `InfoBusItemRequestedEvent` and sends it to all data producers on the bus, indicating the *dataItemName* and the *consumer* that requested it. The consumer can specify its preferred *flavors* or just say `null`.

Each data item supplied by a producer is stored in an array, which is returned to the caller. If no producers offer the requested data item, this method returns `null`.

3.9 The `InfoBusEvent` class

This is the base class for `InfoBus` events used for a rendezvous to provide a data item from a producer to a consumer. Subclasses are defined for each event type for the purpose of determining the event type via the Java “instanceof” operator.

```
public String getDataItemName()
```

This is an accessor that allows an event handler to look at the data item name to see if it can produce or use the named data item.

3.10 The `InfoBusItemAvailableEvent` class

This event is sent on behalf of a data producer to announce the availability of new data to all data consumers that have joined a given `InfoBus` instance. A producer creates and broadcasts the event by calling `InfoBus.fireItemAvailable()`. Because the constructor is package access, the event cannot be created directly by an application.

```
public Object requestDataItem(InfoBusDataConsumer consumer, DataFlavor[] flavors)
```

This method can be called by a consumer to request a data item announced by way of the `InfoBusItemAvailableEvent`. The method creates an `InfoBusItemRequestedEvent` and sends it directly to the producer that announced the item. The producer returns a reference to the item. When *flavors* is not null, it specifies an array of flavors the consumer can use. The producer may decide not to return an item if it cannot provide it in one of these *flavors*.

```
public InfoBusDataProducer getSourceAsProducer()
```

This method returns a reference to the source of the event, i.e. the event handler of the producer that generated the `InfoBusItemAvailableEvent`. The source of available events is always an `InfoBusDataProducer`. `Event.getSource` returns an `Object` reference to the producer. The consumer can use the reference to the producer to uniquely identify the producer of an announced item.

```
public DataFlavor[] getDataFlavors()
```

This method allows a consumer to consider the type of information being announced as available before requesting a data item. It returns a reference to array of `DataFlavor` objects that describe the formats the producer can provide either in the data item itself, or by way of `Transferable.getTransferData()`. If this method returns null, it means the producer did not specify the `DataFlavors` in announcing this data. See “DataFlavors and describing data items” in this chapter for more information.

3.11 The `InfoBusItemRevokedEvent` class

This event is sent on behalf of a data producer to announce the revocation of a previously announced item. It is used by consumers (who should release their reference to the item if they hold it) and controllers (who may wish to update a list of currently available items).

The event is created and broadcast by calling `InfoBus.fireItemRevoked()`. Because the constructor is package

access, the event cannot be created directly by an application.

```
public InfoBusDataProducer getSourceAsProducer()
```

This method returns a reference to source of the event, i.e. the event handler of the producer that generated the `InfoBusItemRevokedEvent`. The source of revoked events is always an `InfoBusDataProducer`.

3.12 The `InfoBusItemRequestedEvent` class

This event is sent on behalf of a data consumer to find a named data item it would like to receive. For example, when an applet is starting, it cannot know whether a given data item has been announced, so it asks for the item by calling one of the find methods in `InfoBus`, which generate this event. Because the constructor is package access, the event cannot be created directly by an application.

```
public void setDataItem(Object item)
```

This is an accessor which the data producer uses to set a data item it is supplying in response to the request event. If the source of this `RequestedEvent` is an `InfoBusSecureConsumer` (forthcoming; see Appendix A), the call to `setDataItem()` will also call the `SecureConsumer`'s `setDataItem()` method to permit the `SecureConsumer` to perform permission checks and determine trustworthiness of the responding producer.

Note that `setDataItem()` is a write-once method: if the data item in the event is non-`null`, it cannot be overwritten. The field will be `null` [writable] when the `RequestedEvent` is first delivered to a producer.

```
public Object getDataItem()
```

This is an accessor which is used by the `InfoBus` or a data controller to get a reference to the data item previously set by a data producer via `setDataItem()`. If no producer responded to the event, calling this method will return `null`. The method is also used in the implementation of `InfoBus.findMultipleDataItems` to get each data item available from the data producers on a given `InfoBus` instance.

```
public InfoBusDataConsumer getSourceAsConsumer()
```

This method returns a reference to the source of the event, ie the event handler of the consumer that generated the `InfoBusItemRequestedEvent`. The source of request events is always an `InfoBusDataConsumer`.

```
public DataFlavor[] getDataFlavors()
```

This method exposes the `DataFlavors` that the consumer prefers, as a hint to producers that can supply data in more than one format. If this method returns `null`, the consumer did not provide any `DataFlavor` preferences when it requested the event. The consumer may specify Mime types in the order it prefers, including `InfoBus` and standard Mime types. A special Mime string "x-InfoBus/AnyAccess" indicates that a consumer will accept any type of `InfoBus` access interface available for the item. Flavors are a hint to the producer, which is not required to consider the requested `DataFlavors` in supplying the data item. See also "DataFlavors and describing data items" in this chapter.

3.13 The `InfoBusEventListener` interface

This interface extends `java.util.EventListener` and `java.beans.PropertyChangeListener` to serve as a base class for the data producer and data consumer interfaces. Figure 3-2 shows the class hierarchy for these interfaces.

The event listeners must be registered with the `InfoBus` after joining it in order to receive events; this is accomplished by calling `InfoBus.addDataProducer` or `addDataConsumer`, as appropriate for the type of event listener interface. An object that serves as both consumer and producer would add itself via both add methods. The listener should be added during the applet's `start()` method (or its moral equivalent) and removed during the applet's `stop()` method in order to reduce event traffic when the browser switches to a different page.

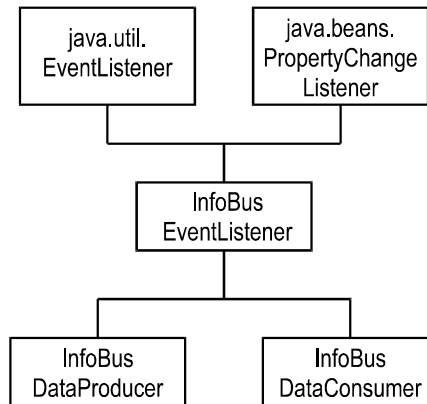


Figure 3-2: Interface hierarchy for InfoBus event listeners

```
public void propertyChange(PropertyChangeEvent event)
```

This method is called whenever the member associated with this producer or consumer has its “InfoBus” property changed by a means other than calling the `leave` method, for example when a builder calls `InfoBusMember.setInfoBus()` to force it to talk to a different bus. The method isn’t really a part of `InfoBusEventListener`, rather it is inherited from `PropertyChangeListener`. The suggested implementation includes:

- Check `event.getPropertyName()` is “InfoBus” and `event.getSource()` is your parent `InfoBusMember`.
- If the `event.getOldValue()` is not `null`, call `event.getOldValue().removeDataProducer()` to stop listening to the old InfoBus instance.
- If the `event.getNewValue()` is not `null`, call `event.getNewValue().addDataProducer()` to listen for events from the new InfoBus instance.

3.14 The InfoBusDataProducer interface

This interface, which extends `InfoBusEventListener`, is implemented by classes that wish to serve as a data producer. A class that implements this interface should be registered with the InfoBus via `addDataProducer()` during the applet’s `start()` method (or the moral equivalent if not an applet), and removed via during the applet’s `stop()` method.

```
public void dataItemRequested(InfoBusItemRequestedEvent event)
```

This method is called by the InfoBus class on behalf of a data consumer that is requesting a data item by name. The suggested implementation:

- check the data item name (obtained via `event.getDataItemName()`) to see if it is an item that can be supplied by this producer. If not, return.
- [in Java 2 Platform] optionally, call `AccessController.checkPermission()` to determine permissions to decide whether to provide the item to the caller.
- create an instance of the data item, or get a reference if it already exists, and set it via `event.setDataItem()`.

3.15 The InfoBusDataConsumer interface

This interface, which extends `InfoBusEventListener`, should be implemented by a class that wishes to serve as a data consumer. The class should be registered with the InfoBus via `addDataConsumer()` during the applet’s `start()` method (or the moral equivalent if not an applet), and removed during the applet’s `stop()` method.

```
public void dataItemAvailable(InfoBusItemAvailableEvent event)
```

This method is called by the InfoBus class on behalf of a data producer that is announcing the availability of a new data item by name. A consumer that obtains a data item from a producer should be prepared to release it when the producer announces that the item is being revoked via `InfoBusDataConsumer.dataItemRevoked()`. The suggested implementation:

- Optionally, call `AccessController.checkPermission()` to determine permissions in deciding whether to request the item from the producer.
- Check the data item name (obtained via `event.getDataItemName()`) to see if the item is wanted. If not, return.
- Get a reference to the data item by calling the `event.requestDataItem()` method.
- If desired, and if a `DataItemChangeManager` is present, set a `DataItemChangeListener` on the data item.

```
public void dataItemRevoked(InfoBusItemRevokedEvent event)
```

This method is called by the `InfoBus` class on behalf of a data producer that is revoking the availability of a previously announced data item. A consumer that is using this data item should release it upon receiving this notification. The suggested implementation:

- Check the data item name (obtained via `event.getDataItemName()`) and the producer sending the event (obtained via `event.getSourceAsProducer()`) to see if this is an item held by this consumer. If not, return.
- Remove any change listeners set on this data item.
- Release all references to the data item held by this consumer.

3.16 The Proxy Listener classes

The `InfoBus` specification requires a reference to a data producer or data consumer, representing the source of an event, to be returned by the `getSourceAsConsumer()` and `getSourceAsProducer()` calls in the various `InfoBus` event classes. The specification also requires a reference to the owner of a data item to be returned by `DataItem.getSource()`. The purpose of these references is to uniquely identify the source of the event or data item, and is expressed as a reference of type `InfoBusDataProducer` or `InfoBusDataConsumer`. The unique identifier allows components to distinguish events or data items, in case more than one data item with the same name is present in a system.

The references also provide an opportunity for trouble from an errant component. Suppose that the source Object that implements one of these event listeners also happens to implement `InfoBusMember` or support properties for a Bean. Any component can introspect on this reference. If `InfoBusMember` is present, the recipient could call its `setInfoBus()` method, for example passing a null reference. This would effectively force the source of an event or data item off the `InfoBus` it previously joined.

We recommend the use of a simple technique to eliminate the opportunity for such problems. An `InfoBus` component creates a separate event listener to serve as its *proxy* for handling events, and registers it as its event listener. When one of the methods to obtain a reference to the source of an event or data item is called, it gets a reference to the proxy listener rather than the parent class. The parent is protected because the proxy Object has nothing of any particular interest that can be introspected.

The `InfoBus` makes this implementation easier by providing classes that serve as the proxy listeners in reusable form. To use a proxy listener, the parent simply creates an instance of the class, passing a reference to itself to the constructor. The parent is required to implement the corresponding listener interface (i.e. `InfoBusDataProducer` or `InfoBusDataConsumer`), because the proxy listeners will call back to methods on these interfaces for the handling of events, but the parent does *not* register itself as a listener. The sample applications included with the `InfoBus` distribution archive illustrate the technique, and code can be cut and pasted from the samples into your application.

3.16.1 The `InfoBusDataProducerProxy` class

`InfoBusDataProducerProxy` is the proxy listener class used by producer components.

```
public InfoBusDataProducerProxy( InfoBusDataProducer parent )
```

The *parent* parameter is a reference to the Object that will handle `InfoBusItemRequestedEvents` and `PropertyChangeEvents` received by the producer proxy listener.

```
public void dataItemRequested( InfoBusItemRequestedEvent event )
```

The `dataItemRequested` method is called by `InfoBus` or a data controller to deliver an `InfoBusItemRequestedEvent`

to this component. The implementation in the proxy class simply delegates the handling of the *event* to its parent class.

```
public void propertyChange( PropertyChangeEvent event )
```

The `propertyChange` method is called by InfoBus or a data controller to deliver an `PropertyChangeEvent` to this component. The implementation in the proxy class simply delegates the handling of the *event* to its parent class.

3.16.2 The `InfoBusDataConsumerProxy` class

`InfoBusDataConsumerProxy` is the proxy listener class used by consumer components.

```
public InfoBusDataConsumerProxy( InfoBusDataConsumer parent )
```

The *parent* parameter is a reference to the Object that will handle `InfoBusItemAvailableEvents`, `InfoBusItemRevokedEvents`, and `PropertyChangeEvents` received by the consumer proxy listener.

```
public void dataItemAvailable( InfoBusItemAvailableEvent event)
```

This method is called by InfoBus or a data controller to deliver an `InfoBusItemAvailableEvent` to this component. The implementation in the proxy class simply delegates the handling of the *event* to its parent class.

```
public void dataItemRevoked( InfoBusItemRevokedEvent event)
```

This method is called by InfoBus or a data controller to deliver an `InfoBusItemRevokedEvent` to this component. The implementation in the proxy class simply delegates the handling of the *event* to its parent class.

```
public void propertyChange( PropertyChangeEvent event )
```

The `propertyChange` method is called by InfoBus or a data controller to deliver an `PropertyChangeEvent` to this component. The implementation in the proxy class simply delegates the handling of the *event* to its parent class.

4. Data items

4.1 Overview

Data items are any Java Objects passed by reference from a producer to a consumer by way of a request event, and any sub-items when collection interfaces are used. The InfoBus API defines a data item transfer object as an Object for maximum flexibility and compatibility with the JDK Collection classes. The InfoBus API defines several interfaces to add InfoBus-standard functionality to the data items:

- The `DataItem` interface provides descriptive and identifying information about the data item itself.
- The `DataItemChangeListener` interface manages `DataItemChangeListener`s from consumers.
- The `DataItemView` interface provides methods to manage a view associated with an item.
- The `awt.data-transfer.Transferable` interface provides an alternate access mechanism for data in another standard format.
- A variety of standard *access interfaces* can be implemented by a data item to provide application-independent access methods for consumers to navigate and retrieve data from the item.

A consumer can examine a data item using the `instanceof` operator (or catch a cast exception) to discover whether a given interface is available on the item. For example, the consumer can find out whether change notifications are available on a given data item by testing for an `instanceof DataItemChangeListener`.

4.2 Example: A simple data item

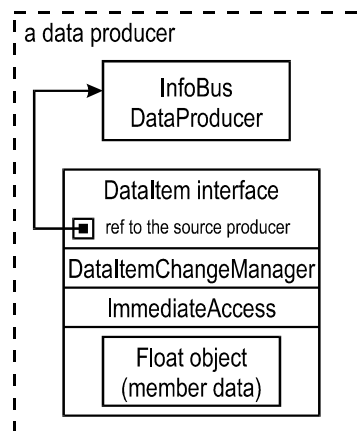


Figure 4-1 - A simple data item offering a Float value with change listener support

Data items can be a single-valued Object wrapper using the `ImmediateAccess` interface. Figure 4-1 illustrates the “`MonetaryDataItem`” from the `SimpleProducerBean` sample application. It implements a data item to identify and describe the data. It implements a `DataItemChangeListener` to accept listeners from consumers that want change notifications. The `Float`, which represents a dollar value, is a member data item, which the consumer can access using methods provided by the `ImmediateAccess` implementation.

A simple data item class definition looks like this:

```
public class MonetaryDataItem
    implements ImmediateAccess, DataItem, DataItemChangeListener
{
    protected Float m_value;
    // methods for DataItem
    // methods for DataItemChangeListener
    // methods for ImmediateAccess
};
```

To access the Float object contained in the MonetaryDataItem instance, the ImmediateAccess.getValueAsObject() method can be called. The reference to the Float is returned, allowing calls to methods on that object, as described in java.lang.Float. The ImmediateAccess interface also defines methods to return a string renderings of the data. The SimpleConsumerBean example uses ImmediateAccess.getPresentationString() to getting a formatted currency string from the producer on the basis of a specified Locale.

For many data items, the data object could be a part of the inheritance hierarchy, in which the class declaration above would extend the data object. This is possible when the data object is not declared as a final class. In such cases, getValueAsObject() simply returns this.

Note that this example does *not* embrace our recommendation that the data be separate from the access object to allow fast creation of data items, the way the next example does.

4.3 Example: A spreadsheet data item

SimpleProducerBean and SimpleConsumerBean are basic examples of producer and consumer applet/Beans. In real-world examples, data items will often contain more interesting data structures, such as a collection of other data items, using various collection interfaces to provide rich structuring of a complex data set.

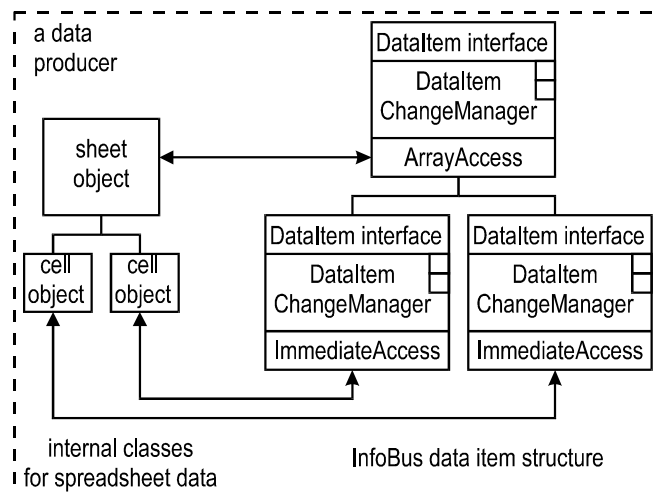


Figure 4-2 - InfoBus objects for a spreadsheet producer that has two cells

Figure 4-2 shows InfoBus objects in a spreadsheet producer, which provides access to a collection of cells by way of the ArrayAccess interface. The getItemByCoordinates() method returns an ImmediateAccess item to provide access to a given cell's data. The spreadsheet offers change notifications at both the sheet and cell level. A spreadsheet producer might also implement a Collection to provide access to various ranges of cells (not shown in the picture).

Spreadsheet data structures generally contain a lot of information that are used for internal purposes and would not be provided to consumers, for example a formula used to calculate the value of a cell. They can also be large, and contain many cells and ranges. This has two important implications in implementing the model.

First, applications of this size will generally not carry copies of their data in the various data items they expose, because it is time-consuming and wasteful of resources. Instead, an access interface serves as a proxy for accessing the data from the internal representation of the sheet, and carries a means of getting the data (e.g., a reference to the cell in the internal representation) rather than a copy of the data.

Second, it is generally inefficient to create data items for all cells when the ArrayAccess data item is created. Instead, ImmediateAccess data items for cells should be created on demand and released when no longer needed.

4.4 The DataItem interface

The DataItem interface provides identifying and descriptive information about a data item. Well-behaved InfoBus producers always implement this interface on the top-level data item being exchanged. For multi-level data items, such

as a Collection of data items, we recommend the implementation of `DataItem` for items at every level. This may not always be possible, thus consumers should be prepared to use it if it is there, and degrade gracefully otherwise.

```
public Object getProperty(String propertyName)
```

Returns a property or metadata information about the item, as indicated by *key*. One key is recommended for `DataItems` at the top-level: the “Name” property must provide the data item name that was used for announcement or request in the rendezvous process. This does not apply to nameless `DataItems`, ie those below the rendezvous point of a hierarchical data item. Support for other properties is optional; `null` should be returned for unsupported properties. Property names should not contain the ‘*’ character.

```
public InfoBusEventListener getSource()
```

This method returns a reference to the creator of this data item. This method should return a reference to the `InfoBusEventListener` (usually an `InfoBusDataProducer`) used for rendezvous of this item as their source.

Data items can also be supplied by a consumer to the producer, for temporary use in changing mutable data items (to provide to the producer a means of accessing the new value), in which case the source of the temporary item is an `InfoBusDataConsumer`.

`null` is not a permissible return value from this method.

```
public void release()
```

This method allows a producer to know when a consumer has finished using a data item. Consumers are required to call this method when they are about to drop their last reference to an item or anything obtained directly or indirectly from the item, including subitems and transfer data (from `Transferable.getTransferData()`). Producers may optionally manage resources for data items in their implementation of this method.

Producer and consumer requirements for this method are discussed in “Requirements for releasing data items,” later in this chapter. Some special considerations for `release()` for data items that implement `RowsetAccess` are discussed in Chapter 5.

4.5 The `DataItemView` interface

Producers may implement `DataItemView` to optimize the management of a view of the contents of a particular subset of records. The view represents the window of data that is currently visible to the consumer.

For example, a consumer of an object that implements `ScrollableRowsetAccess` and `DataItemView` can paint the cells in a grid view of a subset of its rows. As the view is scrolled, the items in the view change as new rows appear. Without the use of this interface, the view can be populated by changing the current row of a `ScrollableRowsetAccess` to get values to be displayed for each row in the view, but this can be time-consuming.

The `ViewStart` property indicates the number of the row in the row set that is seen as the first row of the view. There is no relationship between the current row of the rowset and the `ViewStart`; it is possible to scroll the view without affecting the current row, or change the current row without scrolling the view.

It is possible for a view to contain fewer rows than specified in `getView()`. Similarly, when scrolled to the end, the view may end up with fewer rows than were originally requested for `viewSize`. A similar situation can occur when rows are deleted from a rowset. In these cases, the `ArrayAccess` obtained from `getView()` must indicate the number of rows actually in the view from `dimension[0]` are returned by `getDimensions()`. Attempts to access items beyond this dimension must cause `IndexOutOfBoundsException` to be thrown.

This use of this interface is optional: producers can implement it or not as they choose; consumers may use it or not if it is present.

```
public int getViewStart()
```

Returns the current value of the `ViewStart` property.

```
public void setViewStart(int absoluteRow)
```

Sets `ViewStart` to *absoluteRow*. The *absoluteRow* should be greater than or equal to zero.

```
public void scrollView(int relativeAmount)
```


Changes `ViewStart` by *relativeAmount* relative to its current position. *relativeAmount* is negative to move backwards, or positive to move forward.

```
public ArrayAccess getView(int viewSize)
```

Returns a two-dimensional `ArrayAccess` object that represents a row and column view with *viewSize* rows. The array must be two-dimensional, corresponding to row and column numbers in the view, respectively. The `ArrayAccess` should be read-only; an attempt to set a new value must throw `UnsupportedOperationException`.

Sub-items returned by this `ArrayAccess` must be `ImmediateAccess` items that correspond to the current view to provide standard access to the values in each sub-item. If the `ViewStart` property changes, the values returned by items in the array change so that the view maps to a different range of rows in the row set.

For example, if `DataItemView` were implemented on a `RowsetAccess` object, and an `ArrayAccess` was obtained by calling this method, and `ViewStart` is 0, getting the item at coordinate [1,2] of the array returns the item at row 2, column 3 in the row set. If the consumer calls `scrollView(5)`, `ViewStart` changes to 5, and the value of item [1,2] changes to be the value at row 7, column 3 in the row set.

4.6 Using the `awt.datatransfer.Transferable` interface

The `Transferable` interface can optionally be implemented on any data item at any level. The `Transferable` mechanism provides an alternative to the access interfaces for accessing one or more JDK-standard formats for a data item. Using this mechanism it is possible to achieve essentially a dynamic clipboard implementation. The `Transferable` object exposes the data items `DataFlavors`, as described in

java.sun.com/products/JDK/1.1/docs/api/Package-java.awt.datatransfer.html

When the producer wishes to share a `Transferable` implementation that can work for more than one data item, it can carry a reference to the implementation, and delegate `Transferable` method calls directly to that object.

When implementing `Transferable.getTransferDataFlavors()`, the returned array must include only those MIME types for data that can be accessed via `Transferable.getTransferData()`.

Note that when `Transferable.getTransferData()` is used to get data in a particular data flavor, the reference handed back counts as one of the references that must be released prior to calling `DataItem.release()` at any point above the `Transferable` object.

4.7 Synchronization and locking

`InfoBus` access interfaces do not provide an explicit locking API. Producers can lock a given class instance in the implementation of access methods using synchronized methods or blocks as needed.

4.8 Requirements for releasing data items

Some data items require a critical resource to be allocated by the producer. For example, a data access component may allocate a system resource on a database server that must be released when it is no longer needed. Such producers will typically track the consumers that have requested data items associated with the resource, for example by count or by handing out separate instances of the data item access object, and release the resources when the last consumer indicates that it has finished using the resource. The `DataItem.release()` method is designed to provide an indication to the producer when a given consumer has finished using a data item.

Consumer requirements

Consumers are required to call the `DataItem.release()` when they are finished using any object obtained directly or indirectly from the data item, including objects returned by `Transferable.getTransferData()`. After a consumer calls this method, it must not make any further calls on the `DataItem` or its sub-items, and should drop its reference (for example, if it's member data, set that member reference to `null`). When `release()` is called for any `DataItem`, it means that the consumer is finished with the item at that level and all subitems it may have. When a consumer passes around a data item references among various objects it manages, it must track its use of the references so that it knows when the last reference is dropped.

The consumer may optionally look for sub-items that implement `DataItem` and release these when they are no

longer needed. This is a good idea for large, multi-level collection data items.

Producer requirements

DataItem is required for all top-level (rendezvous) data items, and is optional for all sub-items in a collection hierarchy. The producer is required to provide an implementation of `release()` for each object that implements DataItem. However, `DataItem.release()` is only a hint to the producer, and it can use any strategy it chooses for managing associated resources, or provide an empty implementation to ignore the hints.

A producer may decide to manage resources at the top-level data item, or at all levels of the collection hierarchy, or not at all, according to its own requirements. In a multi-level collection hierarchy which contains a DataItem at more than one level, calling `DataItem.release()` at any level means that that node and all nodes below it are released; calling `release()` on the top-level DataItem releases the entire data item for the consumer that called it.

When a producer supports the release of resources, it must do so in such a way that when one consumer calls `release()`, it will not affect other consumers who hold a reference to any part of the same data item. Of course, because a producer manages the lifetime of a data item, it can revoke it at any time it chooses, but ideally it aims to minimize disruption to consumers. In any case, when a producer does release resources associated with a data item, it should always send a `DataItemRevokedEvent` to listeners of the item, and if it is a top-level item, it should also send an `InfoBusItemRevokedEvent` by calling `InfoBus.fireRevoked()`.

Example

Consider Figure 4-3. There's a data item called A which was provided to the consumer via a rendezvous. A implements a DataItem and a Collection interface. Collection A has two sub-items, Collection B1 and B2, which each implement DataItem. Collection B1 has one sub-item, an ImmediateAccess C, that implements Transferable but does not implement DataItem. Suppose the consumer has requested references to B1, B2, C, and a data transfer object T obtained from `C.getTransferData().B1.release()` should be called when all references to C and T have been dropped, and the consumer plans to drop B1 immediately after calling `B1.release()`.

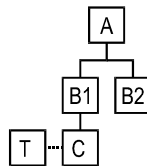


Figure 4-3 - example for describing `release()` rules

`A.release()` should be called when all references to B1, B2, C, and T have been dropped, and the consumer will drop A immediately after calling `A.release()`. Note that calling `A.release()` implies that all sub-items are also released, so when all references to sub-items have been dropped, `B.release()` need not be called before calling `A.release()`.

4.9 Access Interfaces

In the first chapter we suggested that data items suitable for exchange on the InfoBus are those that might be useful to more than one consumer and might be available from more than one producer. Another way of considering this is: can the data be represented by one of the standard access interfaces defined here? Although private access methods can be used with items exchanged on the InfoBus, it defeats the purpose of providing a standard means of exchange between arbitrary components.

The InfoBus access interfaces, when implemented on a data item, allow the consumer to access the producer's data in a standard, application-independent fashion, without knowing the details of the internal representation of the data the way the producer sees it. A producer should implement as many standard access interfaces as possible, to support the greatest variety of consumers, and richer function of those consumers. In general, the access interfaces are not mutually exclusive; depending on the nature of the data, it might make sense to implement all of them.

With the access interfaces presented in this specification, together with the JDK Collection classes, most types of data items, with any arbitrary structure, should be representable and accessible in a standard fashion. Though we encourage these as the standard, we also recognize the need for future compatibility, thus Object is the data type

specified by `get/setDataItem()` in the `InfoBusItemRequestedEvent`. `Object` is also the data type for objects managed by the JDK Collection classes. See also “The use of JDK Collection interfaces for InfoBus access” in this chapter.

4.10 Mutability

Producers establish a policy of when, if ever, data can be changed by consumers. The policy may vary depending on the consumer’s permissions. Most InfoBus access interfaces define methods for changing the data items. If a producer chooses not to support changes to a data item, whether for any consumer or on the basis of examining a particular consumer, it can throw an exception in place of making the change. This style is in keeping with the mutability design for JDK Collections.

When using Java 2 Platform, the producer may call `AccessController.checkPermission()` to determine permissions during the handling of a method that can change a data item. The producer decides how permissions will be used in this case.

We recommend that producers accept an `ImmediateAccess`, if present in the argument to `setValue`-type methods, as the preferred source of a *new Value*, but also allow for producers to accept other types of `Objects` that make sense for the application. This applies to all access interfaces defined in this InfoBus Specification as well as the similar methods in interfaces defined for JDK Collections. `ImmediateAccess` provides the producer with methods needed to determine the new value. It can attempt to parse a string representation, or copy an `Object` if it is of a suitable type, or use the value from an `Object`.

The mutable data item must implement all methods that change data items, including collections, in a way that is thread-safe, for example by using a synchronized method or block. If the data item supports `DataItemChangeEvents`, it must distribute change events due to changes before exiting the synchronized code region. This means a consumer that is changing a data item can rely on seeing the event for the change it initiated before its call to change a data item returns.

4.11 The `ImmediateAccess` interface

A data item implements this interface to allow data values to be retrieved directly from calls to methods on this interface, returning an immediate rendering of the data as a `String` or `Object`. The interface is also convenient for wrapping final `Objects`, which cannot be passed with additional interfaces via a subclass, as in the example showing a data item housing a `Double` at the beginning of this chapter.

`ImmediateAccess` is strongly recommended for data items that are not collections, because they provide common renderings as a `String` or `Object`, independent of the type of data the item represents. This makes life easy for data consumers, who can simply use these strings for representing the object to the user, without knowing any more about the nature of the `Object`.

`ImmediateAccess` can also be used to supply a user-presentable string identifying the collection as a whole. This may be different from the advertised data item name, which is obtained by a method in the `DataItem` interface.

```
public String getValueAsString()
```

This method returns an unformatted representation of the data item. For example, if the item represents a dollar value, the string may return just the digits. There is no requirement that the returned string be the same as `getValueAsObject().getString()`.

```
public String getPresentationString(java.util.Locale locale)
```

This method returns a formatted representation of the data item appropriate for the specified *locale*. For example, if the item represents a dollar value and the locale indicates the United States, the string may be formatted with a dollar sign, commas, and a decimal point. If *locale* is `null`, the consumer is asking for the producer’s default locale.

This method may throw `UnsupportedOperationException` to indicate that it does not support the specified *locale*.

```
public Object getValueAsObject()
```

This method returns a reference to the `Object` that is “wrapped” in this `ImmediateAccess`. The type of the

Object is implementation dependent. A consumer that accesses the Object directly may interrogate it to discover its type, or may examine the MIME type (if available) for this purpose. A producer may choose not to expose an Object by returning `null`.

Consumers should not attempt to modify a mutable Object returned from this call, because the producer cannot know when the Object is changed, and thus cannot send data item change notification events. Producers can protect themselves from such consumers by handing out a *copy* of the Object that holds the current value, so that changing the copy does not affect the value the producer maintains.

```
public void setValue(Object newValue) throws InvalidDataException
```

This method sets a *newValue* for the immediate data in this object. We recommend that all producers accept an ImmediateAccess, if present, as the preferred source of a *newValue*, but also allow for producers to accept other types of Objects that make sense for the application. A producer should not change the type of Object housed in an ImmediateAccess, only its value. See also “Mutability” in this chapter.

The producer’s implementation must obtain the new value from the argument object before returning, rather than saving a reference to it, because the caller may change its value after the return. In obtaining the *newValue* from the argument, the provider’s implementation may need to make a deep (recursive) copy of *newValue* to get all values changed by the consumer, for example when *newValue* is a collection of subitems.

If the item supports change notifications, the producer should notify listeners after the value has changed, but before returning from `setValue`. Such change notifications look the same as if the producer had changed the value itself. See “Mutability” in this chapter for a discussion of synchronization requirements in changing a value.

UnsupportedOperationException should be thrown when the underlying data source does not support changes from any consumer (e.g., read-only). `java.security.AccessExceptions` should be thrown when a given consumer does not have permission to change the data. `java.lang.IllegalArgumentException` should be thrown if *newValue* is a type that is not recognized by the producer for representing the new value. `InvalidDataException` should be thrown if the new value is invalid for this field.

4.12 The ArrayAccess interface

Data items that implement the ArrayAccess interface are collections of data items organized in an n-dimensional array. ArrayAccess objects are bounded in size; that is, their dimensions can be obtained at any time.

Essential to the notion of an array is that you have random access (without significant performance penalty) to any element of the array. Almost all the other forms of data could be modeled as a (degenerate) form of an array, but often the notion of unpenalized access to any element does not hold true.

Methods that specify coordinates throw `ArrayIndexOutOfBoundsException` when the coordinates are invalid.

```
public int[] getDimensions()
```

The number of integers in the returned array indicates the number of dimensions in the object implementing the ArrayAccess, and the value of each integer indicates the number of elements in the ArrayAccess in that dimension — e.g., a return value of {5,6,7} indicates a 5 x 6 x 7 three-dimensional ArrayAccess.

```
public Object getItemByCoordinates(int[] coordinates)
```

This method retrieves an individual data item from an ArrayAccess by way of its index. Retrieval of a data item via `getItemByCoordinates()` should not affect the cursor for other access interfaces on the object that implements ArrayAccess. Indexing in each dimension is zero-based; that is, *coordinates*[i] range from 0 to (`getDimensions()[i] - 1`), to agree with Java arrays.

`null` must be returned to indicate an empty cell.

```
public void setItemByCoordinates(int[] coordinates, Object newValue)  
    throws InvalidDataException
```

This method sets a new value for an item at the indicated coordinates in an ArrayAccess. Setting a data item via this method should not affect Iterators for other access interfaces on the object that implements ArrayAccess. Indexing in each dimension is zero-based; that is, *coordinates*[i] range from 0 to (`getDimensions()[i] - 1`), to agree with Java

arrays. We recommend that all producers accept an `ImmediateAccess` for setting the newvalue. See also “Mutability” in this chapter.

`UnsupportedOperationException` must be thrown when the underlying data source does not support changes from any consumer. `java.security.AccessExceptions` must be thrown when a given consumer does not have permission to change the data. `Java.lang.IllegalArgumentException` must be thrown if *newValue* is a type that is not recognized by the producer for representing the new value. `InvalidDataException` must be thrown if the new value is invalid for this field.

```
public ArrayAccess subdivide( int[] startCoordinates, int[] endCoordinates)
```

This method returns an `ArrayAccess` which is a subset of the `ArrayAccess` on which it was called, with coordinates in the new subset readjusted to start at zero. For example, a data set arranged as rows and columns can be divided into arrays representing individual columns or rows. The *endCoordinates* must be equal to or greater than the *startCoordinates* for all dimensions. The method throws an `ArrayIndexOutOfBoundsException` on an out-of-bounds access.

4.13 The ReshapeableArrayAccess interface

Producers that allow a consumer to change the shape of an array, including changing the dimensions, the number of dimensions, or inserting or deleting all elements in a particular dimension (i.e. “delete column”), implement the `ReshapeableArrayAccess` interface to indicate their willingness to change the shape and provide methods used for this purpose. `ReshapeableArrayAccess` extends `ArrayAccess` to add these methods.

Producers that supply data items with the `ReshapeableArrayAccess` are required to implement the `DataItemChangeManager` on those data items, and to fire the `DataItemShapeChangeEvent` when modifications are made to the array by way of the methods described below.

```
public void setDimensions(int[] newDimensions) throws IllegalArgumentException
```

This method can be called to change the dimensions of an existing array. Such changes involve appending or removing one or more new columns, rows, planes, etc from the end(s) of the array . Any new cells that are added in this operation must be empty cells (e.g., calling `getItemByCoordinate()` on a new cell returns `null`).

The number of integers in *newDimensions* indicate the requested number of dimensions in the object implementing the `ArrayAccess`, and the value of each integer indicates the requested number of elements in the `ArrayAccess` in that dimension.

All producers that implement this interface must be able to change existing dimensions. Producers may or may not support a change to the number of dimensions; those that do not support a change in number of dimensions throw the `UnsupportedOperationException` for such attempts.

`IllegalArgumentException` must be thrown if any of the specified *newDimensions* are less than one (zero for any dimension would result in an array with no cells).

For example, given a 2x2 array, passing *newDimensions* as [2,3] adds a new row. Calling this method with [2,2,3] either adds two new 2x2 planes to form a three-dimensional array of 2x2x3, or if the producer does not support changes to the number of dimensions, `UnsupportedOperationException` is thrown.

```
public void insert(int dimension, int position, int count)  
    throws IllegalArgumentException
```

This method is called to insert additional cells, columns, rows, planes, etc, in a specified *dimension*. *dimension* specifies which entry will be affected in the array of dimensions for this array (as returned by `getDimensions()`).

When adding cells in a given *dimension*, the extents of other *dimensions* are unchanged. For example, in a two-dimensional array, when inserting a new row of cells, the number of cells added is the extent of the column dimension.

position is a zero-based cell offset in the specified *dimension*. *position* can range from zero up to the maximum extent for the specified *dimension*. When *position* is zero up to the extent for the indicated *dimension*, cells are inserted before the indicated cell. When *position* is equal to the extent for the indicated *dimension*, cells are

appended at the end of the array.

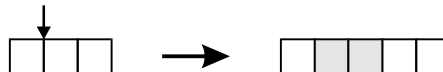
count specifies the number of cells to be inserted in the specified *dimension*; it must be greater than zero. The total number of cells inserted with the operation is *count* times the extents of each dimension other than the specified *dimension*. For example, when *count* is one for a 3x3 array, a new row or column of three cells is inserted. See the examples below for more information.

`IllegalArgumentException` must be thrown if *dimension* is greater than the count of entries from the dimensions returned by `getDimensions()`. It must be thrown if *position* is less than zero or greater than the count of cells in the specified *dimension*. It must be thrown if *count* is not greater than zero.

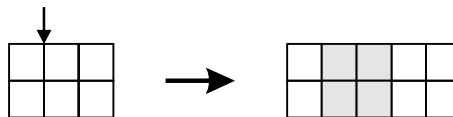
Finally, for the special case of an n-dimensional array, where $n > 1$ and one or more of the dimensions other than the one specified by *dimension* has a zero extent (thus, the array has no cells), it is not possible to insert new cells, and `IllegalArgumentException` must be thrown. In this case the `setDimensions()` method can be used to change the dimensions. Note that an array with a zero extent can be the result of calling the `delete()` method when *position*=0 and *count* is set to the maximum extent for the given *dimension*.

The required behavior is easiest to understand when considering simple pictorial representations. In the following diagrams, the white squares represent cells of an `ArrayAccess` that already exist, and may be empty or not. The gray cells represent new, empty cells added as a result of the indicated operation. Note that the use of the terms columns, rows, and planes to refer to particular dimensions is arbitrary, and is only used for describing the diagrams for these examples.

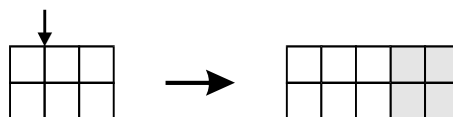
Example 1: Inserting cells in a single-dimension array. Before insertion, `getDimensions()` returns [3]. After calling `insert(0, 1, 2)` to insert two cells before cell 1, new cells are added as shown, and calling `getDimensions()` returns [5].



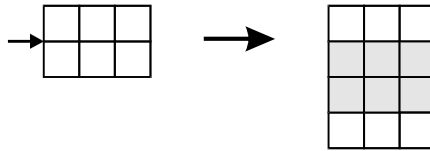
Example 2: Inserting columns in a two-dimension array. Before insertion, `getDimensions()` returns [3,2]. After calling `insert(0, 1, 2)` to insert two columns before column 1, new cells are added as shown, and calling `getDimensions()` returns [5,2].



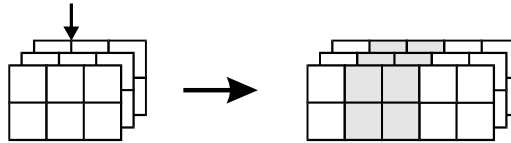
Example 3: Appending columns in a two-dimension array. Before insertion, `getDimensions()` returns [3,2]. After calling `insert(0, 3, 2)` to append two columns (note that the *position* here is equal to the count of cells in dimension 0, meaning “append”), new cells are added as shown, and calling `getDimensions()` returns [5,2].



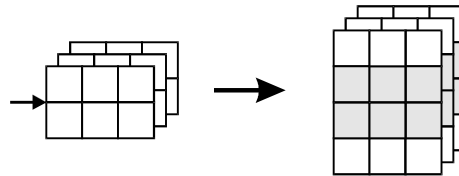
Example 4: Inserting rows in a two-dimension array. Before insertion, `getDimensions()` returns [3,2]. After calling `insert(1, 1, 2)` to insert two rows before row 1, new cells are added as shown, and calling `getDimensions()` returns [3,4].



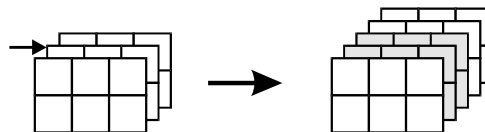
Example 5: Inserting columns into three-dimension array. Before insertion, **getDimensions()** returns [3,2,3]. After calling **insert(0,1,2)** to insert two columns before column 1, new cells are added as shown in all three planes, and calling **getDimensions()** returns [5,2,3].



Example 6: Inserting rows into three-dimension array. Before insertion, **getDimensions()** returns [3,2,3]. After calling **insert(1,1,2)** to insert two rows before row 1, new cells are added as shown in all three planes, and calling **getDimensions()** returns [3,4,3].



Example 7: Inserting planes into three-dimension array. Before insertion, **getDimensions()** returns [3,2,3]. After calling **insert(2,1,2)** to insert two planes before plane 1, new cells are added as shown in two new planes, and calling **getDimensions()** returns [3,2,5].



```
public void delete(int dimension, int position, int count)
    throws IllegalArgumentException
```

This method is called to delete cells, columns, rows, planes, etc, in a specified *dimension*. *dimension* specifies which entry will be affected in the array of dimensions for this array (as returned by **getDimensions()**).

position is a zero-based cell offset in the specified *dimension*. When *position* is zero up to the extent for the indicated dimension, deletions start at the indicated cell.

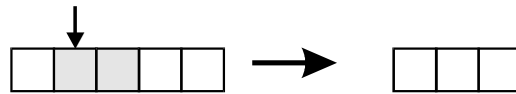
count specifies the number of deletions in the specified *dimension*; it must be greater than zero, and less than the remaining cells of the indicated *dimension* starting with the indicated *position*.

IllegalArgumentException must be thrown if *dimension* is greater than the count of entries from the dimensions returned by **getDimensions()**. It must be thrown if *position* is less than zero or greater than or equal to the count of cells in the specified *dimension* starting with the indicated *position*. It is thrown if *count* is not greater than zero, or if *position* plus *count* is greater than the current extent in the indicated *dimension*.

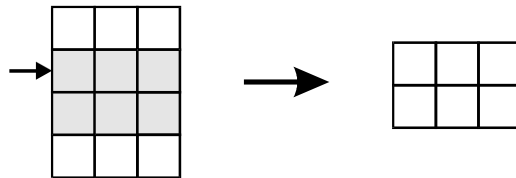
Some examples of the required behavior:

Example 1: Deleting cells in a one-dimension array. Initially, **getDimensions()** returns [5]. After calling

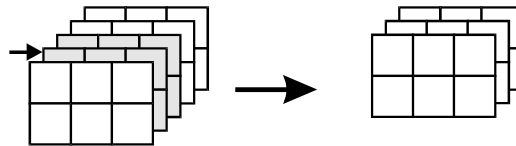
delete(0,1,2) to delete two cells starting with cell 1, cells are removed as shown, and calling **getDimensions**() returns [3].



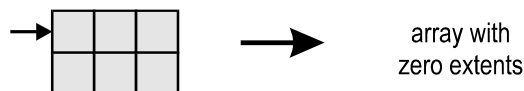
Example 2: Deleting rows in a two-dimension array. Initially, **getDimensions**() returns [3,4]. After calling **delete**(1,1,2) to delete two rows starting with row 1, cells are removed as shown, and calling **getDimensions**() returns [3,2].



Example 3: Deleting planes in a three-dimension array. Initially, **getDimensions**() returns [3,4,5]. After calling **delete**(2,1,2) to delete two planes starting with plane 1, cells are removed as shown, and calling **getDimensions**() returns [3,4,3].



Example 4: Deleting entire contents of a two-dimension array. Initially, **getDimensions**() returns [3,2]. After calling **delete**(1,0,2) to delete two rows starting with row 0 (i.e., all rows), cells are removed as shown, and calling **getDimensions**() returns [0,0]. Calling **delete**(0,0,3) to delete three columns starting with column 0 (i.e., all columns) has the same result. In both cases, empty cells can be added only by calling **setDimensions**() with an array where both extents are non-zero.



4.14 Using JDK Collection interfaces for InfoBus access

In addition to the InfoBus data access interfaces described in this chapter and in Chapter 5, the interfaces defined for JDK Collections are also allowed for a standard means of sharing collections of data and collection hierarchies. Collection, Map, List, and Set interfaces are all suitable container contracts.

Because these interfaces are not defined in JDK 1.1, an interim release of the JDK Collections package is available from JavaSoft; its package name will be com.sun.java.util.collections.

In Java 2 Platform, the implementations provided in java.util can be used if convenient, or the interfaces can be used with a custom implementation, for example, to provide standard access for an existing private data structure).

We recommend that implementations of the methods to set new values use InfoBus access interfaces as the source of the new value (or container of values). See also “Mutability” in this chapter.

JDK Collections are described at <http://www.javasoft.com/products/jdk/preview/docs/guide/collections/>.

4.15 ArrayAccess and JDK Collections

When ArrayAccess and any of the JDK Collection access interfaces are implemented on the same data item, there may or may not be a relationship between the order of accessing elements using an Iterator or ListIterator and the indices of an ArrayAccess. When using the standard implementations of the collections, it may not be convenient to provide indexed access in an efficient manner.

When using the JDK Collection interfaces as a public contract for a private implementation along with ArrayAccess, we recommend that the right-most integer in the dimensions array be the index that changes most frequently when an object is iterated. For example, an ArrayAccess that returns {5, 4, 3} as its dimension array is a 5 x 4 x 3 array, and successive calls to Iterator.next() would return the following elements from ArrayAccess:

```
0, 0, 0
0, 0, 1
0, 0, 2
0, 1, 0
0, 1, 1
etc.
```

4.16 Tree data items

InfoBus does not define a specific access interface for implementing a tree. Trees should be implemented by using one of the JDK Collection interfaces recursively, e.g. creating a Collection that contains Collection objects, and so on.

4.17 Requirements for InfoBus-compliant data items

The task of creating an InfoBus-compliant data item includes making a decision on which of the interfaces to implement. While the InfoBus API requires only that a data item be an Object, there are additional requirements for an InfoBus-compliant data item. Note that the ‘top-level’ item refers to the one handed out via the rendezvous mechanism, which may have sub-items. Data items that are members of a collection data item are referred to here as “sub-items.”

- The spec requires that top-level data items implement DataItem. We recommend implementing DataItem for sub-items whenever possible.
- DataItemChangeManager is recommended for all data items where it makes sense, including sub-items of multi-level data items. When present, the manager is required to manage listeners and send notifications to them for all changes.
- An InfoBus-compliant data item is required to provide at least one of the standard access interfaces for top-level data items. We recommend the use of these interfaces for all sub-items. Standard access interfaces include those defined in this spec, and those defined by the JDK Collections specification.
- We recommend that methods used to set a value in a mutable data item accept an ImmediateAccess, if present, as the source of the new value; other Objects can be accepted at the discretion of the producer.

5. Database access interfaces

5.1 Overview: database access for the InfoBus

The relational access model

In many cases including Relational Database Management Systems, data is organized into (or can be returned as) tables containing rows and columns. Each row has the same number of columns, and each column is homogenous — within a column, the data is of a particular datatype, or `null`. A table may have no rows. The data is usually managed by a server program which controls all access to the data.

To retrieve data from such a source, the client composes a query (typically in a dialect of SQL), submits it to the database server, and receives a result set of rows, or rowset, in return. It is then possible to determine the “shape” of the rowset (the number of columns returned and their names and datatypes). There may be no data rows in the rowset.

To send data to such a source, or modify the data, `INSERT`, `UPDATE` and `DELETE` operations are supported. These return a success indicator and the number of rows affected (this may be zero), but not a result set. Other operations are also usually supported, including such things as creating and deleting tables, granting and revoking access, and storage management. These operations return a success indicator but not a result set or number of rows affected.

Database access and the InfoBus

While tables in a database and the rowsets returned from retrieval queries could be modeled as InfoBus `ArrayAccess` data items, this is not a natural match for the following reasons:

- The number of rows and columns is not known ahead of time and can be expensive to determine, so `ArrayAccess.getDimensions()` cannot always be supported;
- A very large number of rows may be returned
- The column names and datatypes may not be known ahead of time and it may be necessary to discover this at runtime.

To solve these problems we provide the `RowsetAccess` interfaces. This family of interfaces can be used for constructing a data item in addition to or instead of other access interfaces defined in the previous chapter.

When the data is provided by a remote server, a data access component (DAC) can be constructed as a producer that provides `RowsetAccess` data items. The DAC serves as a translator between the remote source and the local consumers of the data. Figure 5-1 illustrates the use of a DAC for serving data to local consumers.

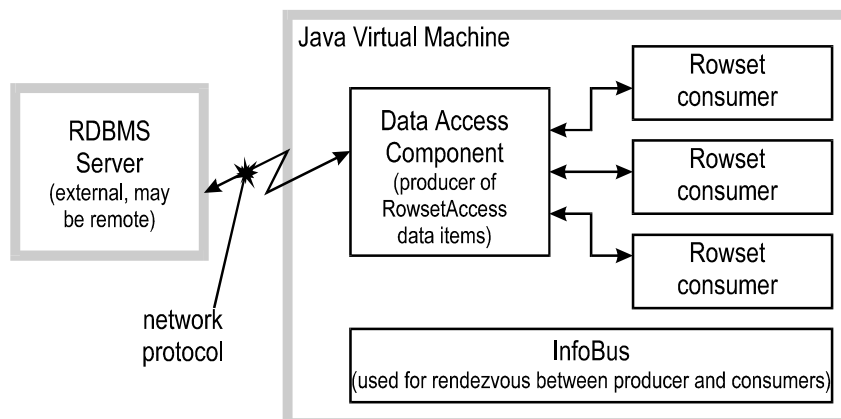


Figure 5-1: Using a data access component as a producer of `RowsetAccess` data items

5.2 The `RowsetAccess` model compared to other access interfaces

The `RowsetAccess` interfaces represent a different model from the access interfaces in Chapter 4, in that the

contents of a RowsetAccess item change as the cursor is moved. This difference reflects the orientation of an external, potentially huge data store. Note that the data is not “in” the data access applet or bean, but in another data store, usually on another machine. The interfaces in chapter 4 are more oriented toward internal data, i.e. data which is “in” an applet or bean running in the local environment such that the total size is known and all the data is immediately available.

Also, the RowsetAccess interfaces extend the use of the DataItemChangeEvent in two ways. First, if there are any change listeners on a RowsetAccess item, a DataItemChangedEvent is emitted when the row cursor changes. Second, data items are used to represent column values, and as the row cursor changes, these data items are modified by the data producer and change notifications sent to any listeners. While this is a standard InfoBus mechanism, in this case it is the data producer itself which is changing the values of the items representing columns.

Even with these differences, it might make sense to implement both RowsetAccess and ArrayAccess for some data items. For example, if a query results in a set of a hundred rows, the data access component (producer) may choose to make it available via both interfaces.

5.3 DataBase Cursor and RowsetAccess design

To process a retrieval query, a database server may do extensive work on behalf of the client, using available indexes wisely, constructing temporary tables when appropriate, obtaining and releasing physical locks on data and index pages, etc. The server typically maintains bookkeeping structures as it returns the result rows, and the current row is presented to the client via a “cursor.” Servers free up resources as soon as possible in order to serve more clients more efficiently, so generally only one cursor is supported per result set. While some servers support backward scrolling cursors, only forward scrolling cursors are guaranteed.

In support of the database notion of a cursor, RowsetAccess implements a slightly different model for data items compared to those described in Chapter 4. Whereas it looks like a collection of records (rows), when the consumer obtains a row of information, the row actually contains the information for the record at the current cursor position. When the cursor changes, the contents of a row also changes. A change notification is available that tells the holder of a row item when its contents changed because of cursor movement. Also, when consumer changes the cursor, holders of an item of the current row all see their contents change. Finally, it is not possible to watch for value changes on an arbitrary row, only on the current row.

5.4 The RowsetAccess interface

Data items that implement the RowsetAccess interface are collections of rows obtained from a data source, usually a relational database server. The RowsetAccess interface contains methods to discover the number and type of columns, to get the next row, to obtain column values, and to insert, update and delete rows.

Initially the rowset is positioned before the first row, and it is necessary to call nextRow() to obtain each row.

Metadata methods

These methods return information about the rowset rather than the data itself.

```
public int getColumnCount()
```

Returns the number of columns in the rowset.

```
public String getColumnName(int columnIndex)
    throws IndexOutOfBoundsException
```

Columns are numbered from one. Given the one-based *columnIndex*, returns the name of the column if available, else null. For example, calculated columns may have no name. See java.sql.ResultSetMetaData.

```
public int getColumnDatatypeNumber(int columnIndex)
    throws IndexOutOfBoundsException
```

Columns are numbered from one. Given the one-based *columnIndex*, returns the column’s SQL type using the encoding in java.sql.Types. See java.sql.ResultSetMetaData. For producer specific datatypes, this should be java.sql.Types.OTHER.

```
public String getColumnDatatypeName(int columnIndex)  
    throws IndexOutOfBoundsException
```

Given the one-based *columnIndex*, returns the column's data source specific type name. See `java.sql.ResultSetMetaData`. For producer specific datatypes, this should be the package and subpackage qualified name of the Java class used to represent the datatype, such as "com.yourorg.data.SpecialDataType".

Cursor movement

```
public boolean next() throws SQLException, RowsetValidationException
```

Advances the row cursor to the next row. Returns `true` if there is a next row, `false` otherwise. It is valid for a rowset to contain zero rows, so the first call to `nextRow()` may return `false`. If modification of the data is supported, moving the cursor may submit a pending change to the database.

When a rowset's cursor is moved, if the rowset has any `DataItemChangeListeners`, the data producer calls the listener's `rowsetCursorMoved()` method with a `DataItemChangeEvent` object. For more information, see Chapter 6, Monitoring changes to data items.

Note that only one data consumer should call `nextRow()`; if two or more consumers each get the same data item implementing `RowsetAccess` and both use `nextRow()`, each can miss some of the data.

```
public int getHighWaterMark()
```

Returns the total number of rows known to the data producer. The data producer should not throw an exception for this method.

In the simplest case, the data producer merely counts the rows as it fetches them. In more sophisticated cases, the data producer may be able to obtain information from a middle tier which fetches rows in chunks.

```
public boolean hasMoreRows()
```

Returns a conservative indication of whether more rows exist in the set. Specifically, a returned value of `false` indicates that the last row has been accessed. A returned value of `true` indicates that further rows *may* exist.

Simple data providers may return `true` when actually on the last row and then return `false` after detecting they have fetched the last row. Data providers for sophisticated backends may be 100% accurate. Sophisticated consumers can avoid an extraneous attempt to retrieve non-existent rows in the case that `false` has been returned.

Data retrieval methods

InfoBus data access components must use the standard mapping between JDBC types and Java types (see JDBC specification, Section 8 — Mapping SQL Data Types into Java. The JDBC spec can be found at <http://java.sun.com/products/jdbc/index.html>). For a data item obtained as a column value, when obtained as an `Object` (for example, by calling `ImmediateAccess.getValueAsObject()`), the `Object` must have the same actual type as the Java type corresponding to the column's JDBC SQL type.

```
public Object getColumnItem(int columnIndex)  
    throws IndexOutOfBoundsException, SQLException
```

Given the one-based *columnIndex*, returns a data item which can be used to obtain the current value of the specified column. This is usually an `ImmediateAccess` item. The value changes as `nextRow()` is called, that is the data item tracks the current row, and if the column item implements `DataItemChangeManager`, `DataItemChangeEvents` are generated.

This method throws `IndexOutOfBoundsException` if the column index is invalid.

```
public Object getColumnItem(String columnName)  
    throws ColumnNotFoundException, DuplicateColumnException, SQLException
```

Given the *columnName*, returns a data item which can be used to obtain the current value of the column. This is usually an `ImmediateAccess` item.

As for `getColumnItem(int columnIndex)` above, the value changes as `nextRow()` is called; that is, the data item tracks the current row, and if the column item implements `DataItemChangeManager`, `DataItemChangeEvents` are generated.

This method is useful instead of the column number version above when many columns are returned and the order of the columns changes over time, but the names of the columns do not change.

This method throws an exception if the specified column is not present in the RowsetAccess object, or if more than one column with a matching name is found.

We define both flavors of getColumnItem() to return a data item which tracks the value of the specified column in the current row. By definition, this succeeds if the column name or number is valid, but throws an exception as described above if the column number or name is invalid.

Since the data item returned is (usually) an ImmediateAccess item, the data consumer must still call getValueAsObject(), getValueAsString(), or getPresentationString() to obtain the value. We specify that if no row is available, getValueAsObject(), getValueAsString(), and getPresentationString() should throw an exception.

If the data item returned is not an ImmediateAccess item, the data consumer must still invoke other methods to obtain values in the column. In relational databases, a column value is normally a scalar, but in principle RowsetAccess could be used to return non-scalar column data from other types of data sources.

Thus one need only obtain a data item for each desired column once. This is more efficient even for the simple use pattern.

Data modification methods

These methods insert, update, and delete rows. If the data item does not support a particular operation, the owner can throw UnsupportedOperationException (a runtime exception and thus not listed explicitly in the **throws** clause of these methods.) The owner may also throw java.security.AccessControlException if the caller does not have permission to change the data, or InvalidDataException if the new data is not valid.

Note that column values may also be modified via ImmediateAccess.setValue() if a data item has been obtained for the column.

After a row is changed, cursor movement may cause the row to be submitted to the underlying data store, and this may cause an exception. However, not all data providers submit changes on cursor change. The flush() method may be used to explicitly submit a changed row to the database.

`public void newRow()` throws SQLException, RowsetValidationException

Creates a new, empty row and sets the row cursor to this row. Since this changes the row cursor, this may propagate a changed row to the back end, and this may throw an exception.

`public void setColumnValue(int columnIndex, Object object)`
throws SQLException, RowsetValidationException, IndexOutOfBoundsException

Sets the value of the specified *columnIndex* in the current row to the supplied value. *columnIndex* is the one-based column number. This is used both to update existing rows and to supply values for new rows. We recommend that all producers accept an ImmediateAccess, if present, as the preferred source of a *newValue*, but also allow for producers to accept other types of Objects that make sense for the application.

`public void setColumnValue(String columnName, Object object)`
throws SQLException, RowsetValidationException,
ColumnNameNotFoundException, DuplicateColumnException

Sets the value of the specified column in the current row to the supplied value. *columnName* must identify exactly one column. We recommend that all producers accept an ImmediateAccess, as the preferred source of a *newValue*. See also “Mutability” in Chapter 4.

`public void deleteRow()` throws SQLException, RowsetValidationException

Deletes the current row.

`public void flush()` throws SQLException, RowsetValidationException

Explicitly submits changes in the rowset to the underlying data store.

`public void lockRow()` throws SQLException, RowsetValidationException

Requests a row level lock on the current row, if supported by the backend and the data producer. The method does nothing if it is not supported.

Normally each row change is an implicit transaction, and the lock is released by moving to a different row. If an explicit transaction has been started by way of a call to `DbAccess.beginTransaction()`, the lock is released during the processing of `DbAccess.commitTransaction()` or `DbAccess.rollbackTransaction()`.

Determining the mutability of the data source

Data repositories support different combinations of retrieval, insert, update and delete. Some are read only, some allow all operations, and some allow other combinations such as read and insert but not delete or update.

The following methods allow the data consumer to determine which operations may be attempted. Note that a particular operation may fail due for other reasons such as access control, integrity constraints, or network connection problems.

```
public boolean canInsert()
```

Returns `true` if inserting new rows is allowed, `false` otherwise.

```
public boolean canUpdate()
```

Returns `true` if modifying the items in all columns in the existing rows is allowed, `false` otherwise.

```
public boolean canUpdate(String columnName)
    throws ColumnNotFoundException, DuplicateColumnException
```

```
public boolean canUpdate(int columnNumber) throws IndexOutOfBoundsException
```

Returns `true` if modifying the items in the specified column is allowed, `false` otherwise.

```
public boolean canDelete()
```

Returns `true` if deleting rows is allowed, `false` otherwise.

Special methods

```
public DbAccess getDb()
```

This method returns a `DbAccess` item representing the database associated with the `Rowset`. This returns `null` if the `DataProducer` does not support the `DbAccess` interface. For more information, see the `DbAccess` interface below.

5.5 The ScrollableRowsetAccess interface

This interface extends `RowsetAccess`, and represents the case in which the data provider can support moving the row cursor backwards and creating multiple cursors.

```
public ScrollableRowsetAccess newCursor()
```

Returns a new `ScrollableRowsetAccess` having the same underlying data but an independent cursor. The new cursor is positioned before the first row. The object returned is a separate data item from the one on which `newCursor()` was called. The new data item has no name (i.e., if it implements `DataItem`, `getProperty("Name")` should return `null`).

```
public void setBufferSize(int size)
```

Asks the data provider to keep the specified number of rows immediately available. The specified *size* is a hint for performance and does not throw an exception if not supported.

```
public int getBufferSize()
```

Gets the buffer size in effect. If `setBufferSize` is not supported, this will be 1.

Cursor movement

In addition to the methods listed here, the `next()` method defined for `RowsetAccess` is also available.

```
public boolean previous() throws SQLException, RowsetValidationException
```

Moves the row cursor to the previous row. Returns `true` if there is a previous row, otherwise `false`.

```
public boolean first() throws SQLException, RowsetValidationException
```

Moves the row cursor to the first row. Returns true if there is a first row, false if the rowset is empty.

public boolean **last()** throws SQLException, RowsetValidationException

Moves the row cursor to the last row. Returns true if there is a last row, false if the rowset is empty.

public boolean **relative**(int *numRows*) throws SQLException, RowsetValidationException

Moves the row cursor forward the specified number of rows (or back if *numRows* is negative.) Returns true if the specified row exists, false otherwise. This can position the cursor before the first row or after the last row.

public int **getRow()**

Gets the row number of the current row.

public int **getRowCount()**

Returns the total number of rows in the rowset. Some data producers will not support this operation without fetching all the rows, and should throw UnsupportedOperationException.

public boolean **absolute**(int *rowIndex*) throws SQLException, RowsetValidationException

Moves the row cursor to the specified *rowIndex*. Returns true if the specified row exists, false otherwise.

5.6 The DbAccess interface

In some cases, the data consumer will wish to control the lifetime of a data item representing a rowset. This can be important if the retrieval query ties up significant resources on the database server, or large numbers of rows are involved, or both. In this scenario, only the consumer knows how long the data item is needed. For example, if a query returns a large number of rows (say one million), if there is only one consumer, and if the consumer is only interested in the first few rows (say one screenful), the data item and the resources on the database server should be released as soon as the consumer reads the first screenful of rows.

Also, in some cases it is more convenient for the data consumer to construct the query and control when it is executed and whether the result is made available to other InfoBus aware components.

Connect and disconnect methods

The Connect and Disconnect methods intentionally mirror their counterparts in java.sql.DriverManager and java.sql.Driver. They are intended for cases in which components other than the data producer need to control the time of connection and disconnection, since a connection to a database can be an expensive resource.

A producer can implement the DataItem interface on RowsetAccess items and DbAccess items to provide the release() method. By supporting release(), the producers may choose, if appropriate, to implicitly disconnect when the last dependency on resources goes away.

public void **connect**() throws SQLException

Attempts to establish a connection to the given database URL. Any required connection arguments such as user ID and password must be defined in the producer. For example, these might be supplied to the producer outside the DbAccess interface via HTML <PARAM>s, JavaBean properties, or incoming InfoBus data items.

public void **connect**(String *url*, String *username*, String *password*) throws SQLException

Attempts to establish a connection to the given database *url* using the supplied *username* and *password*.

public void **connect**(String *url*, Properties *info*) throws SQLException

Attempts to establish a connection to the given database *url* using the connection arguments in *info*. Typically, “user” and “password” properties are required.

public void **disconnect**() throws SQLException

Unconditionally disconnects from the current database. The producer should announce that all data items have been revoked prior to disconnecting, by firing an InfoBusItemRevokedEvent via the InfoBus and by firing a DataItemRevokedEvent on each DataItemChangeManager. Further use of the DbAccess object, except to connect, is undefined. Further use of RowsetAccess/ScrollableRowsetAccess objects associated with the

DbAccess object is undefined.

```
public java.sql.DriverPropertyInfo[] getPropertyInfo(String url, Properties info)
```

This method allows a data consumer to discover what connection arguments the data producer requires to connect to a database. The database is specified by *url*, and a proposed list of connection arguments is specified by *info* (this may initially be empty.)

The resulting array of DriverPropertyInfo objects may be empty if no connection properties are required. Otherwise it is a list of properties, some of which may be required. See java.sql.Driver.getPropertyInfo. In complex cases it may be necessary to call getPropertyInfo() multiple times — the possible or required connection arguments may depend on previous choices.

Query methods

```
public Object executeRetrieval(String retrieval, String dataItemName,  
    String options) throws SQLException
```

This method executes the specified retrieval query and returns the result as an Object. *retrieval* specifies the retrieval query, which is typically a SQL SELECT or a stored procedure which returns a result. If *dataItemName* is not null, the data provider should make the resulting RowsetAccess item available under the specified *dataItemName*.

options provides special instructions to the data provider. This may be null, or a space delimited list of option strings. The producer is not required to honor these requests, but if it does it should use the specified syntax. The producer is not required to honor these requests. When using the functions listed here, the producer should use the strings indicated for those functions. A producer can add other functions as needed.

The following option strings are predefined:

Option string	meaning
---------------	---------

“ArrayAccess”	asks the producer to return an object which implements the ArrayAccess interface.
---------------	---

“Map”	asks the producer to return an object which implements the Map interface
-------	--

“RowsetAccess”	asks the producer to return an object which implements the RowsetAccess interface.
----------------	--

“ScrollableRowsetAccess”	
--------------------------	--

	asks the producer to return an object which implements the ScrollableRowsetAccess interface.
--	--

“PreFetch= <i>n</i> ”	asks the producer to pre-fetch the specified number of rows, where <i>n</i> represents the number. 0 means none, -1 means all.
-----------------------	--

“RowLimit= <i>n</i> ”	asks the producer to fetch no more than the specified number of rows, where <i>n</i> represents the number. 0 means none (for cases where only the resulting column names and datatypes are desired.), -1 means all.
-----------------------	--

The method returns an item implementing RowsetAccess if the operation succeeded. If the operation fails, an SQLException is thrown.

```
public int executeCommand(String command, String dataItemName) throws SQLException
```

This method executes the specified non-retrieval query and returns the count of rows affected, or -1 if this is not applicable. *command* specifies a non-retrieval query, such as SQL INSERT, UPDATE, DELETE, or a stored procedure which does not return a result. If *dataItemName* is not null, it instructs the data provider to make the count of affected rows available as an ImmediateAccess data item. The method returns the number of rows affected. This can be 0 for INSERT, UPDATE, DELETE or equivalent, or -1 if this is not applicable (for commands which do not manipulate rows). If the operation failed, a SQLException is thrown (see java.sql.SQLException).

Transaction methods

By default, changes are implicitly committed when sent to the underlying data store.

Many database servers and intelligent middle tiers support grouping of modifications into transactions such that the group of changes is atomic (either all of them take effect or none of them take effect.) Such transactions are

at the database level, since changes to multiple tables are allowed. Because the capabilities of databases vary, some of these methods are optional and may have no effect, as noted.

```
public void beginTransaction()
```

Do not commit changes when they are sent to the underlying data store. Begin explicit commit mode. The producer should throw `UnsupportedOperationException` if it does not support this method.

```
public void commitTransaction() throws SQLException, RowsetValidationException
```

Performs any database integrity and consistency checks on changes sent to the database since the last `beginTransaction`. If all checks pass, make the changes permanent in the database. Multiple tables may have been modified using multiple rowsets; all such changes are applied. Following the commit, resume implicit commit mode.

```
public void rollbackTransaction() throws SQLException, RowsetValidationException
```

Undo all changes sent to the database since `beginTransaction`. This may affect multiple rowsets. Resume implicit commit mode.

```
public void validate() throws SQLException, RowsetValidationException
```

If supported, performs explicit validation of all changes to the database since the last `beginTransaction()`, without committing them. Multiple tables may have been modified using multiple rowsets; all such changes are validated. Does nothing (and does not throw an exception) if not supported by the data producer.

```
public void flush() throws SQLException, RowsetValidationException
```

If supported, sends all changes made through any rowset to the database. Column, row, cross row, and some cross table integrity and consistency checks may be applied. Does nothing if not supported by the data producer.

5.7 The RowsetValidate interface

Producers may implement the `RowsetValidate` interface to provide a means of validating the contents of a `Rowset` data item. This interface is optional: producers can implement it or not as they choose; consumers may use it or not if it is present.

```
public void validateCurrentRow() throws RowsetValidationException
```

Explicitly validates data in the current row only.

Typically, the producer performs checks that can be done locally without involving underlying data store. For example, this method could check that the value in a column is one of the allowed values or is in a range of values.

```
public void validateRowset() throws RowsetValidationException
```

This method validates data in the current rowset, taken as a set of rows.

Typically, the producer performs checks that can be done locally without involving underlying data store. The checks may involve more than one row. For example, this method could check that the sum of one particular column in the current set of rows does not exceed a particular value.

5.8 Releasing database resources

Chapter 4 discusses the use of `DataItem.release()` to give producers a hint about when it can release a critical resource. When this method is called for database items, the producer must have a policy about how it will behave for items that have been modified but require `commitTransaction` to be called to apply the changes to the database.

We recommend that a producer treat `release()` as having an implied `rollbackTransaction` method call, so that changes are not applied except when explicitly committed by a consumer. If the producer decides to adopt a policy to commit instead, it must be clearly documented so that application designers can plan for this.

6. Monitoring changes to data items

After a consumer acquires a data item from a producer, it can begin to access the data by way of the various access interfaces discussed in the previous two chapters. A consumer may need to monitor changes to the data item, for example to cue it for updating its display. Change notifications are sent by the producer to registered consumers using an event-based scheme. This scheme is similar to but logically separate from the InfoBus events used for rendezvous about data items in order to reduce traffic on the InfoBus.

Four classes and interfaces are defined for the InfoBus mechanism for monitoring data item changes:

- DataItemChangeManager - implemented by the producer on data items for which it is able to notify consumers of changes.
- DataItemChangeSupport - an implementation of DataItemChangeManager that defines methods to manage listeners and fire events that can be used in applications.
- DataItemChangeListener - implemented by a consumer then registered with the producer to receive notifications of changes.
- DataItemChangeEvent - a base class for change events which are sent by the producer to registered consumers via their DataItemChangeListener objects.

In Figure 4-2 we saw a spreadsheet data producer that provided data items for a sheet collection of two cells, along with the relationship between the exposed data items and the internal data representation. In Figure 6-1 below, the same example is illustrated, this time showing the relationship between the producer's data items and the change listeners registered by a consumer application. The consumer is interested in changes occurring to the sheet as a whole, perhaps so it knows when to repaint a graph, as well as changes to one cell in particular, which may be displayed separately.

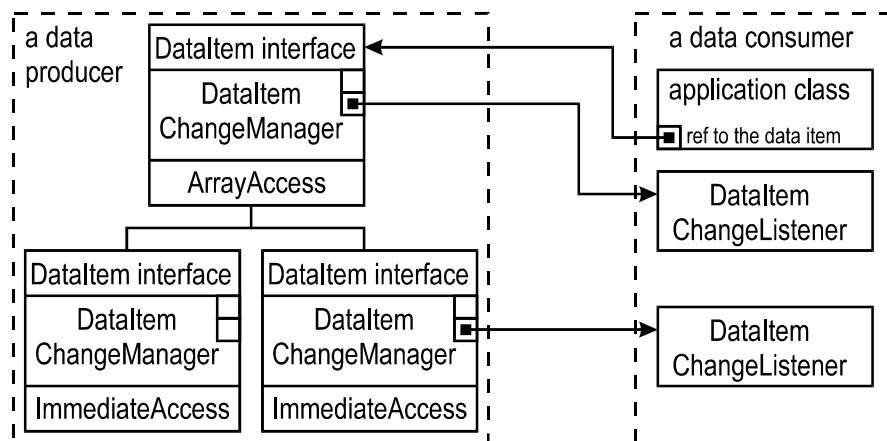


Figure 6-1 - Change managers and listeners in a producer and consumer

In a slightly more complex example, change listeners may be registered at various parts of a data item hierarchy by *different* consumers. To the producer, it looks no different; it does not distinguish which change listeners are provided by which consumer. Change events are “multi-cast” to those listeners that have attached themselves to the data item in no defined order as per section 6.6 of the Java Beans 1.0 specification.

Data producers should be tolerant of exceptions in listeners, and in general should ignore any exceptions thrown by a listener. In particular, an exception thrown by one listener should not prevent other listeners from receiving the event.

6.1 The DataItemChangeManager interface

This interface allows a data item to provide notifications to a consumer when the item has changed, by managing DataItemChangeListeners registered by the consumer for this purpose. The DataItemChangeManager interface is required for data items that implement the ReshapeableArrayAccess interface, and is recommended for all

other data items. We recommend that data items provide change listener support at all levels of a collection hierarchy where it is possible, to offer maximum flexibility to data consumers. This allows the producer to choose the granularity of notifications it wants. In the example above, the consumer can look for changes on any individual cell, or on any in the spreadsheet as a whole.

Data items that implement `DataItemChangeListener` must support registration and deregistration of event listeners as per section 6.5.1 of the JavaBeans 1.0 specification. Specifically, changes to the listener list may take place during notification of all listeners. Accordingly, the listener list should be copied at the beginning of a change notification, and the copy of the list used for the duration of the notification.

When data changes (whether by the producer for its own reasons, or on behalf of a change from any consumer, or in the producer's data source), the producer notifies registered listeners, as described in "Distribution and handling of change events".

```
public void addDataItemChangeListener(DataItemChangeListener listener)
public void removeDataItemChangeListener(DataItemChangeListener listener)
```

These methods form the standard JavaBeans design pattern for an object that emits a `DataItemChangeEvent`. A data consumer interested in continuing updates on a data item will call `addDataItemChangeListener()` to express that interest, and `removeDataItemChangeListener()` when it no longer needs the notifications.

6.2 The `DataItemChangeManagerSupport` class

This class implements `DataItemChangeListener`, and can be used by a producer for any data item. You can use an instance of this class as a member field of your data item and delegate various work to it. Aside from providing the methods to manage listeners, it also has methods for firing each kind of event.

Constructor

```
public DataItemChangeSupport(Object source)
```

The constructor requires references to the data item object that implements the `DataItemChangeListener` interface. When an instance of this class is a member field for a data item, the *source* is a reference to the outer class that contains the instance of this class.

Listener management

```
public void addDataItemChangeListener(DataItemChangeListener listener)
public void removeDataItemChangeListener(DataItemChangeListener listener)
```

These methods add and remove change listeners as requested by a consumer or other InfoBus component.

```
public void removeAllListeners()
```

This method can be used by the producer to remove all listeners that may still be associated with the change manager. It should be called only after announcing that the item is being revoked both by way of `InfoBus.fireItemRevoked` and `DataItemChangeSupport.fireItemRevoked`.

Event-firing methods

Each method below creates an appropriate change event and sends it to all the listeners at that level only. Events should also be distributed to other levels according to the rules specified in "Distribution and handling of change events," later in this chapter.

```
public void fireItemValueChanged(Object changedItem,
    InfoBusPropertyMap propertyMap)
```

This method should be called when an item, usually an `ImmediateAccess`, changes value. The caller indicates the *changedItem* as the one whose value changed. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply `null` for this parameter.

```
public void fireItemAdded(Object changedItem, Object changedCollection,
    InfoBusPropertyMap propertyMap)
```

This method should be called when one or more new items are being added to a collection. The caller indicates the *changedItem* as the one being added, and *changedCollection* as the collection that gained an item. *changedItem* can be null when more than one item is added in the same operation. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply null for this parameter.

```
public void fireItemDeleted(Object changedItem, Object changedCollection,
    InfoBusPropertyMap propertyMap)
```

This method should be called when one or more items are being removed from a collection. The caller indicates the *changedItem* as the one being removed, and *changedCollection* as the collection that lost an item. *changedItem* can be null when more than one item is removed in the same operation. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply null for this parameter.

```
public void fireItemRevoked(Object changedItem, InfoBusPropertyMap propertyMap)
```

This method should be called when an item or collection is no longer available, such as when the data source is going offline. The caller indicates the *changedItem* as the item or collection that is being revoked. Unlike the other events, this event is sent to the data item passed during rendezvous, and to all sub-items in a collection hierarchy. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply null for this parameter.

```
public void fireRowsetCursorMoved(Object changedItem,
    InfoBusPropertyMap propertyMap)
```

This method must only be called for a RowsetAccess object when the rowset's cursor has moved to a different row. The caller indicates the *rowset* whose cursor changed. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply null for this parameter.

```
public void fireItemShapeChanged(Object changedItem,
    InfoBusPropertyMap propertyMap)
```

This method must be called when a data item changes shape. For example, a DataItemShapeChangedEvent should be fired when any of the extents of a ReshapeableArrayAccess, as reported by getDimensions(), change, for example as a result of calling setDimensions(), insert(), or delete(). This method must only be called for data items that implement the ReshapeableArrayAccess interface.

The caller specifies the object that is changing dimensions in *changedItem*, which is always a reference to a ReshapeableArrayAccess object. Producers that wish to supply additional information about the change may do so by supplying a *propertyMap*; producers that do not offer this information should supply null for this parameter.

6.4 The DataItemShapeChangeListener interface

This interface extends the DataItemChangeListener interface to add a new change event delivery method.

```
public void dataItemShapeChanged(DataItemShapeChangedEvent event)
```

Indicates a change in the shape for a data item. For example, a ReshapeableArrayAccess has been changed such that one or more of its dimensions has changed. A reference to the data item that changed can be obtained from the *event*.

6.5 The DataItemChangeListenerSupport class

This class implements DataItemChangeListener and DataItemShapeChangeListener, and can be used by a consumer as a base class for implementing a change listener class. All methods in this class have empty-body implementations. To use this class, the consumer should subclass it and override methods for handling events of interest.

```
public void dataItemValueChanged(DataItemValueChangedEvent event)
```

Default handler for the DataItemValueChangedEvent, which simply ignores the *event*. If the *event* is of interest, the consumer should override this method with one that handles the event.

```
public void dataItemAdded(DataItemAddedEvent event)
```

Default handler for the DataItemValueAddedEvent, which simply ignores the *event*. If the *event* is of interest, the consumer should override this method with one that handles the event.

```
public void dataItemDeleted(DataItemDeletedEvent event)
```

Default handler for the DataItemValueDeletedEvent, which simply ignores the *event*. If the *event* is of interest, the consumer should override this method with one that handles the event.

```
public void dataItemRevoked(DataItemRevokedEvent event)
```

Default handler for the DataItemRevokedEvent, which simply ignores the *event*. If the *event* is of interest, the consumer should override this method with one that handles the event.

```
public void rowsetCursorMoved(RowsetCursorMovedEvent event)
```

Default handler for the RowsetCursorMovedEvent, which simply ignores the *event*. If the event is of interest, the consumer should override this method with one that handles the event. This method will only be called for data items that implement the RowsetAccess interface.

```
public void dataItemShapeChanged(DataItemShapeChangedEvent event)
```

Default handler for the DataItemShapeChangedEvent, which simply ignores the *event*. If the event is of interest, the consumer should override this method with one that handles the event. This method will only be called for data items that implement the ReshapeableArrayAccess interface.

6.6 The DataItemChangeEvent class and event subclasses

DataItemChangeEvent is the base class of all other events described in this chapter. A data item fires a DataItemChangeEvent to all registered DataItemChangeListeners to indicate that a change has occurred. The nature of the change is indicated by the name of each subclass. The easiest way to fire change events is use DataItemChangeSupport, which has a fire method for each type of event.

The listener handles change events by implementing DataItemChangeListener.dataItemChange(). It uses the instanceof operator to distinguish the various change events, and handles or ignores them as required.

The event includes a reference to the data item whose data changed and a reference to the item that manages the listener to which the notification is sent. Note that because the *source* of the change can only be set in the constructor, a separate event instance must be created for each change manager implementation in a multi-level collection hierarchy. This is intentional, for security reasons.

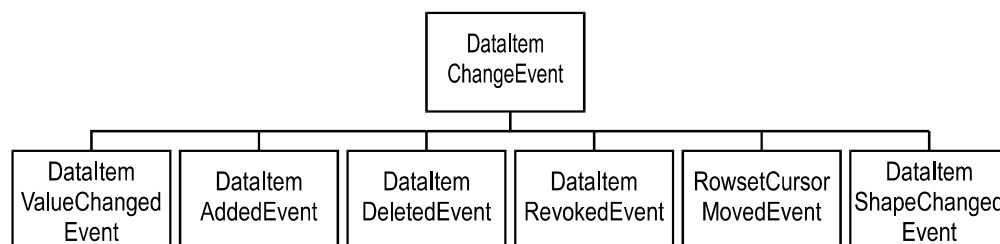


Figure 6-2: Class Hierarchy for DataItemChangeEvents

DataItemChangeEvent base class

```
DataItemChangeEvent(Object source, Object changedItem,  
InfoBusPropertyMap propertyMap)
```

```
DataItemChangeEvent(Object source, Object changedItem, java.util.Map propertyMap)
```

Constructs a DataItemChangeEvent, indicating the item that manages a listener (*source*), the data item that

changed, and an `Map` object that can be used for looking up the values for properties in `getProperty()`. *propertyMap* is optional, and `null` can be used when `getProperty()` is not supported.

The `DataItemChangeEvent` constructor is package access so that only the events defined by the InfoBus spec can be sent.

The first form of the constructor, which accepts an `InfoBusPropertyMap` for the *propertyMap* parameter, will be deprecated in InfoBus 2.0. The second form of the constructor, which accepts a `java.util.Map` for the *propertyMap* parameter, is not present in InfoBus 1.1; this method will be added for InfoBus 2.0.

```
public Object getSource()
```

This method returns a reference to the source data item to which the change event listener is registered. Note that this method is defined in `java.awt.Event`, rather than `DataItemChangeEvent`, which extends it. When a `DataItemChangeSupport` class is used to fire events, the source is the same as the reference passed to its constructor.

```
public Object getChangedItem()
```

This method returns a reference to the data item that changed.

For collection size change and item value change events, this can be the same as the reference returned by `getSource()`, meaning the item that changed is also the one that had the registered change listener. When it is not `null` and is different from `getSource()`, it refers to a sub-item of the one that had the change listener. Finally, it can be `null`, meaning that more than one sub-item has changed in the same operation (for example, delete row).

For the revoked change event, `getChangedItem()` returns a reference to the top-level rendezvous item that is being revoked. The recipient of a revoked change event might not recognize the top-level item under some circumstances — for example, if a lower-level item was published in two overlapping collections. In these cases, the recipient is still obliged to treat the data item identified by `getSource()` as a revoked item.

For a rowset cursor moved event, `getChangedItem()` returns a reference to the `RowsetAccess` data item whose cursor was moved.

```
public Object getProperty(String propertyName)
```

Returns a property or metadata information about the change event. For example, properties can provide information about the nature of sub-items that changed when `getChangedItem()` returns `null` (which indicates that more than one sub-item changed).

Support for properties is optional; if the `DataItemChangeEvent` constructor received `null` as the map reference, `getProperty()` returns `null`. If a reference to an implementation of `InfoBusPropertyMap` was supplied in the change event constructor, this method calls `InfoBusPropertyMap.get()` with the specified *key* and returns the result. `null` is the conventional return value when the specified *key* is not supported as a property name.

DataItemValueChangedEvent, DataItemRevokedEvent, DataItemShapeChangedEvent and RowsetCursorMovedEvent classes

These events extend `DataItemChangeEvent`, overriding the constructor with a public constructor, but add no other methods or data. They also have a public constructor so they can be created using any implementation of `DataItemChangeManager`. Except for having a public constructor, the API is the same as for `DataItemChangeEvent`.

DataItemAddedEvent and DataItemDeletedEvent classes

These classes extend `DataItemChangeEvent` to modify the constructor and add a method that indicates the collection associated with the added or removed item, as well as a public constructor method.

In addition to the methods described below, the `getSource()` and `getChangedItem()` methods from `DataItemChangeEvent` are also available in these events.

```
public DataItemAddedEvent(Object source, Object changedItem,  
    Object changedCollection, InfoBusPropertyMap propertyMap)  
public DataItemDeletedEvent(Object source, Object changedItem,  
    Object changedCollection, InfoBusPropertyMap propertyMap)
```

```

public DataItemAddedEvent(Object source, Object changedItem,
    Object changedCollection, java.util.Map propertyMap)
public DataItemDeletedEvent(Object source, Object changedItem,
    Object changedCollection, java.util.Map propertyMap)

```

Constructs an event, indicating the *source* as the data item that sent the event, the item that was added or removed, the collection that changed, and an Map object that can be used for looking up the values for properties in `getProperty()`. *propertyMap* is optional, and `null` can be used when `getProperty()` is not supported.

The first form of the constructors, which accept an `InfoBusPropertyMap` for the *propertyMap* parameter, will be deprecated in InfoBus 2.0. The second form of the constructors, which accept a `java.util.Map` for the *propertyMap* parameter, are not present in InfoBus 1.1; these methods will be added for InfoBus 2.0.

```

public Object getChangedCollection()

```

This method returns a reference to the collection data item that lost or gained sub-items. For `DataItemDeletedEvent`, can return `null` when the item deleted was a singleton item (i.e., not a member of a collection).

6.7 Distribution and handling of DataItemChangeEvents

This section describes the event classes defined for InfoBus data item change events, when they should be fired, and how consumers should respond to them.

The event classes are subclasses of `DataItemChangeEvent`. An event specifies a source data item, which is the item containing the `DataItemChangeManager`, and the data item that changed. If the reference to the item that changed is `null`, it means that more than one item was changed in the operation, for example by deleting a column of cells or filling a range of cells. Most events are empty subclasses of `DataItemChangeEvent`. `DataItemAddedEvent` and `DataItemDeletedEvent` modify the parent class to offer a reference to the collection that gained or lost items.

Most data items offered at the rendezvous are collections of other data items. A complex data item can have several levels to the collection hierarchy. When data items change, change events are distributed up or down the collection hierarchy, depending on the type of change that occurred, as listed below. When a change occurs, the producer must fire an event to all registered change listeners according to the rules described in this section, in any order that is convenient to the producer. When the rules indicate that a change notification should be sent to a given data item's listener, it means that the event should be sent if the `DataItemChangeManager` is implemented and has registered listeners.

The producer indicates the levels of the hierarchy for which it is willing to provide change notification service by implementing the `DataItemChangeManager` interface at these levels. Although implementations of `DataItemChangeManager` are not required, for maximum flexibility for consumer applications, we recommend an implementation of the interface at all levels where it makes sense. `DataItemChangeSupport` implements `DataItemChangeManager`, and provides methods to fire each type of change event when listeners are present.

Consumers can register a change listener at any level where a `DataItemChangeManager` is present. Listeners determine the nature of a data item change on the basis of the class type of the event they receive. The listener can use `instanceof` or attempt a cast for this purpose. Events can be handled or ignored according to the needs of the consumer, except for `DataItemDeletedEvent` and `DataItemRevokedEvent`, which require the consumer to release references and cease making calls to the data item that changed.

In this section we refer to the “rendezvous item” as the data item provided to a consumer that requests the event, which is often a collection of other data items. Data items that are members of a collection data item are referred to as “sub-items.” “Parent items” refer to any container item in the hierarchy (`ArrayAccess`, `RowsetAccess`, or one of the JDK Collections interfaces) which is the parent of a given sub-item.

DataItemValueChangedEvent

An item is said to change value when calling a method to get its value returns a different result than before it changed. This can result when the underlying data itself has changed, such as when a method is called to change the value. Some items are treated as having a current value, for example the current row of a `RowsetAccess` item; the value is also said to change when the rowset cursor moves to a record that has a different value for a

given column.

When an item changes value, a change notification event should be sent for that item, then for its parent collection, and so on up to and including the rendezvous item. Note that an item can belong to more than one collection, and the collections could have a common ancestor; in this case care must be taken to avoid a redundant item value change notification to the common ancestor. Items that can change value are those that have an immediate value, and are generally not collections. `getChangedItem()` returns a reference to the item that changed, or `null` to indicate more than one item changed in the same operation.

DataItemAddedEvent

When one or more items are added to a collection item, `DataItemAddedEvent` should be sent for each added item and for the parents of the items up to and including the rendezvous point. `getChangedItem()` returns a reference to the item that was added, or `null` to indicate more than one item was added in the same operation. `getChangedCollection()` returns a reference to the collection that gained item(s).

DataItemDeletedEvent

When one or more items are permanently deleted from a collection, an event should be sent for each item and for the parents of the items up to and including the rendezvous point. When a container data item is being deleted, sub-items should fire `DataItemDeletedEvent` with the same rules. This event can also be fired with a singleton data item (i.e., one that is not a sub-item of a container item) is being deleted. `getChangedItem()` returns a reference to the item, or `null` to indicate more than one items were added in the same operation. `getChangedCollection()` returns a reference to the collection that lost the item(s) or `null` if it was a singleton item.

Operations on a `ReshapeableArrayAccess` data item that reduce the number of cells in the array also result in firing `DataItemDeletedEvent` for those cells that are not empty. For example, calling `setDimensions()` when one of the new dimensions is smaller than the corresponding existing dimension will cause the removal of cells; calling the `delete()` method will generally result in the removal of cells as well. For cells being removed that are not empty (i.e. calling `getItemByCoordinate()` would return a non-`null` reference to a data item), the `DataItemChangeEvent` should be fired for each, indicating the reference to the data item that is being deleted; alternatively, one `DataItemChangeEvent` can be fired with `null` as the item that is being deleted, meaning that more than one non-empty cell is being removed. These events must be fired before firing the `DataItemShapeChangeEvent`.

The consumer must cease making calls on this item, release any reference to it, and call `DataItem.release()` on the item.

DataItemRevokedEvent

Sent when an item is being revoked by its producer, i.e. it is no longer available from the producer, for example because the producer's data source is going away. The event should be sent to all listeners for the item and, if it is a container item, to all listeners of any sub-items in the containment hierarchy. This event can also be fired with a singleton data item (i.e., one that is not a sub-item of a container item) is being revoked. This event differs from `DataItemDeletedEvent` in that it the item may be available in the future.

The consumer must cease making calls on this item, release any reference to it, and call `DataItem.release()` on the item.

DataItemShapeChangeEvent

Indicates that the one or more of the dimensions of a `ReshapeableArrayAccess` data item has changed. The event must be sent to listeners of the `ReshapeableArrayAccess` item whose shape has changed, with *changedItem* referring to the `ReshapeableArrayAccess`.

Note that when changes to the shape of this data item results in the removal of one or more non-empty cells, the `DataItemDeletedEvent` must be fired first, indicating the removal of the cells, before sending `DataItemShapeChangeEvent`. See the subsection "DataItemDeletedEvent" above for more details.

RowsetCursorMovedEvent

Indicates that the cursor for a `RowsetAccess` data item has moved to a different row. The event should be sent to listeners of the `RowsetAccess` whose cursor changed.

6.8 Examples of event propagation

Consider Figure 6-3, which is an example of a multi-level data item shown with a few sub-items. Note that ‘Subset’ contains some of the same items as ‘All’. ‘Top-level’ is the rendezvous point.

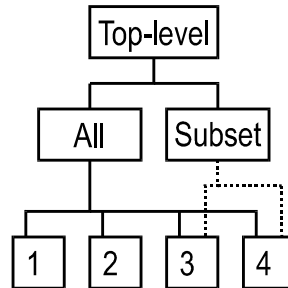


Figure 6-3: A data item hierarchy example with an item in more than one collection in the hierarchy

The following list looks at various changes that are possible in this hierarchy and the way events should be distributed to notify listeners. When we say notification should occur at a given level, it means that it should occur of the item at that level implements `DataItemChangeListener` and has at least one listener, and that all listeners at that level receive the event. Whenever an event is sent to a listener of some data item, the source is a reference to the data item that manages the listener list (is a `DataItemChangeListener`).

- If item ‘3’ changes value, fire a `DataItemValueChangedEvent` specifying ‘3’ as the item that changed on listeners of ‘3’, ‘Subset’, ‘All’, and ‘Top-level’. The listeners of ‘Top-level’ should be notified exactly once.
- If item ‘3’ is deleted, fire a `DataItemDeletedEvent` specifying ‘3’ as the item that changed and ‘Subset’ as the collection that changed on listeners of ‘3’, ‘Subset’, and ‘Top-level’. Next, fire the event specifying ‘3’ as the item that changed and ‘All’ as the collection that changed on listeners of ‘3’, ‘All’, and ‘Top-level’. If instead ‘3’ is removed from ‘Subset’ but still remains in ‘All’, only the first set of notifications are sent, because ‘All’ didn’t change.
- If items ‘3’ and ‘4’ are being deleted in one operation, fire a `DataItemDeletedEvent` specifying each item as the item that changed on the listeners of each item, then fire the event specifying `null` as the item that changed on listeners of ‘All’, ‘Subset’, and ‘Top-level’. Instead of sending `null`, notifications could be sent for each of items ‘3’ and ‘4’ to the parent collections.
- If item ‘2’ is added to the ‘Subset’ collection, fire `DataItemAddedEvent` specifying ‘2’ as the item that changed and ‘Subset’ as the collection that changed on listeners of ‘2’, ‘Subset’ and ‘Top-level’.
- If ‘Top-level’ is being revoked, the producer calls `InfoBus.fireRevokedEvent()` for the item, indicating ‘Top-level’ as the one that changed, then it fires `DataItemRevokedEvent` on listeners of all sub-items, each indicating the sub-item as the one that changed. This continues until all sub-items of ‘Top-level’ at all levels are notified.
- Suppose ‘All’ is a `ReshapeableArrayAccess`. If the `delete()` or `setDimensions()` methods are called such that existing, non-empty cells will be removed, one `DataItemDeletedEvent` must be fired for each non-empty cell, indicating a reference to the cell that was deleted and the `ReshapeableArrayAccess` item that contained it, followed by a `DataItemShapeChangeEvent` sent to listeners of the `ReshapeableArrayAccess`. Alternatively, for convenience, it is permissible to fire one `DataItemDeletedEvent` with `null` as the reference to the cell being deleted (meaning more than one non-empty cell was deleted), followed by a `DataItemShapeChangeEvent` sent to listeners of the `ReshapeableArrayAccess`. In both cases, the rules for propagating the `DataItemDeletedEvent` are the same as those described for that event, above. The `ReshapeableArrayAccess` event must be propagated upward to the parent of the array that changed, and its parent, up to and including the top-level node of the data item.
- For a `ReshapeableArrayAccess` whose dimensions change such that empty cells are being added, the

DataItemShapeChangedEvent should be sent to listeners of the array, to its parent, up to and including the top-level node of the data item.

- For a ReshapeableArrayAccess where more than one dimension is changing as a result of **setDimensions()**, it is possible that cells are being removed from one dimension while being added in another. In this case, only one ReshapeableArrayAccess is sent per call to **setDimensions()** to each level of the data item hierarchy from the array up to the top-level node of the data item.
- Suppose 'All' is a RowsetAccess. When its cursor changes, fire RowsetCursorChangedEvent on listeners of 'All'. Then, for each column item that would return a different value as a result of the new cursor position, fire DataItemValueChanged on listeners of the columns whose value changed relative to the previous row.

6.9 The temporary InfoBusPropertyMap interface

InfoBusPropertyMap is a temporary interface designed to provide a mechanism for use with InfoBus 1.1 components that wish to supply properties on DataItemChangeEvents. To use it, the producer implements the interface and provides a reference to the implementation class in the change event constructor.

When implementing this interface for InfoBus 1.1, an implementation of `sun.com.java.util.collections.Map`, such as `HashMap` may be used, delegating the `get()` method to the class.

In InfoBus 2.0, this class and its method will be deprecated. Constructors that use it will also be deprecated, and new constructors will be introduced that specify `java.util.Map` instead. Support for applications compiled with InfoBus 1.1 will be present by removing the `get()` method from `InfoBusPropertyMap` and adding a clause to have it extend `java.util.Map`.

```
public abstract Object get(Object key)
```

Returns the Object to which the specified *key* is mapped. Returns `null` if the map contains no mapping for this key OR it maps to `null`. *key* must be a String, otherwise `ClassCastException` should be thrown. Property names should not contain the '*' character.

7. Data controllers

7.1 Overview

A data controller is an object that implements the `InfoBusDataController` interface and participates in the distribution of `InfoBusEvents` to consumers and producers on an `InfoBus`. Generally, the data controller is a pluggable module that gets added to an `InfoBus` in order to optimize some aspect of the communication on the bus.

The `InfoBus` supports multiple data controllers on one bus. When consumers on an `InfoBus` make requests for data or when producers make requests that `Available` or `Revoked` notices be sent, the `InfoBus` passes the request on to its registered data controllers by polling each in turn. Any controller polled can indicate to the `InfoBus` that the request should not be processed further by returning `true` from the method used to pass in the request. In this case the `InfoBus` will not poll remaining controllers, and returns the results, if any, to the requester of the action. If no data controller indicates an event is processed, or if no controllers are installed, the event is handled by the `InfoBus` default controller. The order in which the data controllers receive a request can be partially determined by a controller priority that is specified when the controllers are added to the bus. The default controller always has the lowest priority so that it is always last.

A data controller is not directly involved with data item method calls, nor is it involved with data item change events. This interaction is directly between producers and consumers, and the `InfoBus` does not intervene. However, a data controller could intervene on data item methods by keeping a producer's item and supplying a proxy in its place to the consumer.

7.2 Data controller examples

The `InfoBusDataController` interface is very powerful and allows a controller to provide a range of functions that are not included in the basic services already supplied by an `InfoBus`. The following are some of the possible optimizations that a custom controller might perform. A single controller might implement one or many of the functions here (or others we haven't thought of).

The simple priority router. The data controller identifies a subset of the total producer and consumer population, perhaps by inspecting the package to which each belongs, or by detecting the presence of an identifying interface. Messages which originate in this subset are routed first within the subset. For example, a `findDataItem()` sent by a consumer in the subset will first query producers in the subset.

The late-binding controller. The data controller maintains tables of data item names along with the producers who announced them and/or the consumers that requested them. When a new `findDataItem()` is issued, the controller first (or perhaps only) queries producers in the table already associated with that name; likewise for a `fireRevoked()`. Variants on this theme might send the request to all producers if the name is not tabled.

Wildcard support. Similar to late-binding, the controller keeps tables of item names and participants, but supports wildcard use in matching requests to previously announced items.

The voting controller. The data controller receives a `findDataItem()` request, but queries multiple producers even after a response is received, then singles out the best answer for its return result. This concept is probably strongest when a subset of voting-aware beans has been isolated. A variant would use a custom interface on producers which could attach a priority or certainty to the results, allowing a method of identifying the best response.

VM bridge. A data controller can be written to propagate `InfoBus` events, data items, and change events to a partner controller in a different VM, using RMI or other network transport. Such a bridge could be used to provide access to remote processes through the `InfoBus`.

InfoBus traffic monitor. The controller is set to `MONITOR_PRIORITY` so that it sees events ahead of other controllers. It monitors `InfoBus` activity and displays information or writes it to a log for debugging purposes.

7.3 The DefaultController

Each `InfoBus` instance always contains a `DefaultController` which provides standard one-to-all distribution, where

a consumer's request is sent to all producers and a producer's announcements are sent to all consumers. In the absence of any custom controllers, the presence of the DefaultController insures a basic, unoptimized level of operation. The DefaultController has the lowest possible priority and is therefore always the last controller on the request distribution list. If none of the data controllers ahead of the DefaultController indicate that the request has been handled, the DefaultController will get the request and process it.

7.4 InfoBus methods supporting data controllers

Adding and removing data controllers

The following methods, defined in the InfoBus class, allow a data controller to be added to or removed from an InfoBus.

```
public synchronized void addDataController(InfoBusDataController controller,  
      int priority) throws InfoBusMembershipException
```

This method registers the indicated *controller* with the InfoBus, causing it to be added to the list of registered controllers by the indicated *priority*.

The *priority* parameter denotes roughly where in the list of controllers the new *controller* should be placed, and is described in detail in the next section. Once added, a data controller will remain at that priority level until removed. Any unique data controller object may only appear once in the list of controllers on a single InfoBus.

Calling the addDataController() method with a *controller* that is already present on the bus will cause an InfoBusMembershipException to be thrown. Calling this method on a stale InfoBus instance will cause StaleInfoBusException to be thrown.

```
public synchronized void removeDataController(InfoBusDataController controller)
```

The remove method will remove the specified *controller* from the InfoBus. Calling removeDataController() with a *controller* that is not currently listed on the InfoBus has no effect. An InfoBus instance will not remove itself from the virtual machine's set of active controllers if there are any data controllers (or InfoBusMembers, producers, or consumers) still registered on the bus. Therefore, applications which have registered a controller with one of their InfoBuses should remove the controller as they exit, just as they remove their member, producer, and consumer objects.

Firing events from data controllers

Creation and delivery of InfoBusEvents is handled exclusively by the InfoBus class. In order to permit an added data controller to optimize the distribution of events on a bus, the InfoBus provides a set of target-specific event delivery methods, which are described in Chapter 3; see "The InfoBus class: firing events."

The target-specific event firing methods on the InfoBus are versions of the findDataItem() and fireItemAvailable/Revoked() methods that specify a single target or a list of specific targets to which the method should be applied. A data controller may call any combination of these methods when processing a request, but should avoid sending multiple events to the same target for efficiency's sake.

The data controller must call only the target-specific versions of the methods on the InfoBus. The methods that do not specify a target or targets are for the exclusive use of producers and consumers, and are handled by calling the data controllers. Calling them *from* a data controller will cause a loop where the request is again distributed to the data controllers for processing.

7.5 Composition of a data controller

A data controller must implement the InfoBusDataController interface to register with an InfoBus. There are no additional requirements on data controllers in terms of other InfoBus interfaces. That is, a data controller is free to implement InfoBusMember, InfoBusDataProducer, InfoBusDataConsumer, or any combination of these interfaces, but is not required to implement any of them.

For example, one type of controller might be an independent bean which gets and joins an InfoBus of its own accord, in which case implementing InfoBusMember is appropriate (it is a requirement of joining).

Another type of controller is one that is instantiated and controlled by other objects in a larger application -- perhaps a primary producer or consumer in that application. In this case, the controller may only implement the

InfoBusDataController interface; since another object is obtaining and setting the InfoBus in this controller, the controller need not even be an InfoBusMember. Again, it is critical in either case that the controller be properly removed from the bus when other participants are being removed and terminated, so that the InfoBus itself can be freed for garbage collection.

7.6 Data controller priority levels

The *priority* specified when adding a controller is used to determine the insertion order in a linked list of established controllers. This list is traversed from the beginning to the end when processing events, so the priority also determines the order in which a controller will be given a chance to handle events. Because a controller can indicate that an event has been handled, and should not be passed on to other controllers for handling, a lower-priority controller may not see all of the events that a higher-priority controller handles.

The InfoBus class declares six priority levels, with higher integers indicating higher priority, and higher priority controllers receiving requests before lower priority ones. The order of delivery among controllers having the same priority is unspecified. The InfoBus defines the following constants for priority values:

- InfoBus.MONITOR_PRIORITY
- InfoBus.VERY_HIGH_PRIORITY
- InfoBus.HIGH_PRIORITY
- InfoBus.MEDIUM_PRIORITY
- InfoBus.LOW_PRIORITY
- InfoBus.VERY_LOW_PRIORITY
- InfoBus.DEFAULT_CONTROLLER_PRIORITY (reserved for javax.infobus.DefaultController)

The MONITOR_PRIORITY is reserved for data controller objects that need to be aware of all requests that arrive at the InfoBus, and is therefore the highest available priority. However, data controllers that assert MONITOR_PRIORITY are expected to be monitor processes, and not actively participate in event distribution. To enforce this concept, values returned by data controllers having MONITOR_PRIORITY are ignored, and the requests proceed to the data controllers with non-monitor status regardless of whether any such monitors exist.

If a priority is specified during addDataController() that is higher than VERY_HIGH_PRIORITY but not equal to MONITOR_PRIORITY, the object will be treated as having VERY_HIGH_PRIORITY because of the special restriction on MONITOR level controllers.

The DEFAULT_CONTROLLER_PRIORITY is the lowest possible priority, but is reserved for the javax.infobus.DefaultController that is always present in an InfoBus, to insure that DefaultController always handles a request that has not been completed by a previous controller. VERY_LOW_PRIORITY is therefore the lowest generally-available priority level. If a controller is added with a priority value lower than VERY_LOW_PRIORITY, it will have that value adjusted to VERY_LOW_PRIORITY.

7.7 Maintaining producer and consumer lists in a data controller

Each data controller on the InfoBus maintains its own lists of producers and consumers that it wishes to serve, which may include all or a subset of the members on the bus. When the InfoBus passes a request to a data controller, the controller decides whether the request applies to the members on its private lists. If so, it calls one of the target-specific event firing methods on InfoBus to initiate delivery of the appropriate event to the members of its choosing. Conversely, if the request does not apply to the members handled by this data controller, the controller simply returns.

The InfoBus provides a copy of its master lists when a data controller first joins the bus, and then updates its data controllers on changes to the master lists. Because data controllers may only concern themselves with a subset of all participants, some additions to the InfoBus may not be reflected in the local list held by a data controller. However, when an InfoBus producer or consumer indicates that it is leaving the bus, all data controllers which included that participant locally are obliged to remove it from their distribution lists.

Data controllers that handle only a subset of producers and consumers may make such a determination when an add method is called. Note, though, that there is no mechanism for a data controller to re-request the entire

master list from its InfoBus — if a controller is not interested in including a new participant on its private list immediately but might in the future, it is the responsibility of the controller to remember that participant.

```
public void setConsumerList(Vector consumers)  
public void setProducerList(Vector producers)
```

These two data controller methods are called on by the InfoBus to which the controller has been added at the time the controller is joining the bus. This is done in order that the data controller can discover what *producers* and *consumers* were on the bus already when it joined.

```
public void addDataConsumer(InfoBusDataConsumer consumer)  
public void addDataProducer(InfoBusDataProducer producer)  
public void removeDataConsumer(InfoBusDataConsumer consumer)  
public void removeDataProducer(InfoBusDataProducer producer)
```

These four methods are called by the InfoBus on each registered data controller when the InfoBus methods of the same names have been called to add a *producer* or *consumer* to the bus. This allows the controllers to make the appropriate adjustments to their lists.

Data controllers may receive add or remove notifications while in the process of handling a request; for example, delivery of an InfoBusItemRevokedEvent to one consumer may cause that consumer and possibly others to remove themselves from the bus. Data controllers must take care not to accidentally repeat delivery or, worse, skip delivery of events to some participants in such situations.

7.8 Data controller event distribution

“The InfoBus class: firing events” in Chapter 3 defines a set of methods in the InfoBus class that fire events for use by producer and consumer components. The events fired by consumers and producers are generally handled by the creation and distribution of appropriate events; for example, when a producer calls fireItemRevoked it expects an InfoBusItemRevokedEvent to be created and sent to the consumers on the bus.

When an InfoBus receives a request from a consumer or producer, it passes the request to its highest priority data controller. The data controller, in turn, decides whether the request applies to the members on its private lists. If so, it calls the target-specific event firing methods on InfoBus to initiate delivery of the appropriate event to the members of its choosing, possibly collecting returned results. Conversely, if the request does not apply to the members handled by this data controller, the controller simply returns.

The methods used to pass the requests to the data controller have a boolean return value which indicates whether the InfoBus should stop or continue processing the request. A return value of `true` indicates that all processing of this request is complete and that no further data controllers should be consulted; `false` indicates that the processing should continue with the next available controller.

```
public boolean fireItemAvailable(String dataItemName, DataFlavor[] flavors,  
    InfoBusDataProducer producer)
```

This method is called by an InfoBus to pass a *producer*’s request for an ItemAvailable broadcast. A data controller can distribute an InfoBusItemAvailableEvent to any of its consumers by calling the target-specific versions of fireItemAvailable() on the InfoBus. The value of the source parameter from the calling of the data controller’s method should be copied to all target-specific calls, to preserve the identity of the original requester.

The return value indicates whether processing is complete: if `true`, no other data controllers are called regarding this request.

```
public boolean fireItemRevoked(String dataItemName, InfoBusDataProducer producer)
```

This method is called by an InfoBus to pass a *producer*’s request for an ItemRevoked broadcast. A data controller can distribute an InfoBusItemRevokedEvent to any of its consumers by calling the target-specific versions of fireItemRevoked() on the InfoBus. The value of the source parameter from the calling of the data controller’s method should be copied to all target-specific calls, to preserve the identity of the original requester.

The return value indicates whether processing is complete: if `true`, no other data controllers are called regarding this request.

```
public boolean findDataItem(String dataItemName, DataFlavor[] flavors,
    InfoBusDataConsumer consumer, Vector foundItem)
```

This method is called by an InfoBus to pass a *consumer*'s request for the named data item. A data controller uses the InfoBus's target-specific versions of `findDataItem()` to query any of its producers. The value of the *consumer* parameter from the calling of the data controller's method should be copied to all target-specific calls, to preserve the identity of the original requester.

The *foundItem* Vector is passed by the InfoBus as a location for storing a response if one is found. If *foundItem* is not empty when the call completes, the element at 0 in the Vector is taken as the result and passed by the InfoBus back to the consumer. In this case, the `boolean` return value is ignored and no other controllers receive the request. If the *foundItem* Vector is empty after the method completes, the return value indicates whether processing is complete: if `true`, no other data controllers are called regarding this request, and `null` is passed to the requesting consumer.

```
public boolean findMultipleDataItems(String dataItemName, DataFlavor[] flavors,
    InfoBusDataConsumer consumer, Vector foundItems)
```

This method is called by an InfoBus to pass a *consumer*'s request for the named data item. A data controller uses the InfoBus's target-specific versions of `findDataItem()` to query any or all producers it is managing. The value of the consumer parameter from the calling of the data controller's method should be copied to all target-specific calls, to preserve the identity of the original requester.

The *foundItem* Vector is passed by the InfoBus as a location for storing responses if found. If *foundItem* is not empty when the call completes, the elements in the Vector are concatenated by the InfoBus with results from other controllers polled (with elimination of duplicate occurrences of an object). The return value indicates whether processing is complete: if `true`, no other data controllers are called regarding this request.

Although a consumer's `findMultipleDataItems()` request is sent to data controllers, it should only be handled in special cases. The desired behavior of a `findMultipleDataItems()` is that each producer on the bus be queried exactly once for the named data, and the collection of all responses is returned in *foundItems*. This behavior is exactly that performed by the `DefaultController`, and therefore custom data controllers should usually simply defer to the `DefaultController` for handling the find-multiple case, by returning by returning `false` and leaving *foundItems* empty.

In situations where a custom controller decides to handle `findMultipleDataItems()`, there are some special considerations.

- The single-target version of `findDataItem()` should be used to query each producer being managed in turn — the Vector version will stop on the first response and is therefore unsuitable for gathering multiple response data.
- Results returned by the data controllers are concatenated by the InfoBus. The InfoBus will remove redundant responses by eliminating duplicate objects from the concatenated array; however, producers that are queried more than once may return different response objects (based, for example, on the security clearances on inquiring classes, which will include the controllers themselves).

In short, the two safest ways to handle a `findMultipleDataItems()` within a data controller are to either do nothing (rely on the `DefaultController`) or, conversely, to query all producers on the bus and then return `true` to stop further processing. Firing an event to a component then returning `false` to allow handling by other controllers will always result in an event being fired more than once to the same component, and should be avoided.

7.9 Example of event distribution process

Figure 7-1 illustrates the handling of a consumer's request event in a system with four established controllers and five producers. In the diagram, the controllers were added as follows:

- The event monitor controller was added as `MONITOR_PRIORITY`. As the highest priority controller, it sees all events. Because it is passive and does not send events, it does not maintain lists of producers or consumers.

- Controllers 1 and 2 were added at `HIGH_PRIORITY` and `MEDIUM_PRIORITY` respectively. For this example these controllers only manage requests to producers and so do not have lists of consumers, although they could, if they wanted, manage announcements to consumers as well. Controller 1 decided that it would manage producers V and W. Controller 2 decided that it would manage producers X, Y, and Z.
- The default controller was created with `DEFAULT_CONTROLLER_PRIORITY` so that it is always the last controller to handle events. It is created by the InfoBus instance, and has package-level access to the InfoBus's producer list, so it does not keep its own copy. It implements the default rules (all producers see all requests, all consumers see all available / revoke announcements) when none of the other controllers indicate that an event has been handled. Because no other controller handles available and revoke announcements, the default controller handles these events.

Also, the Vector used for the producer list has entries for each of the producers V through Z, but the lines are not drawn in order to keep the diagram readable.

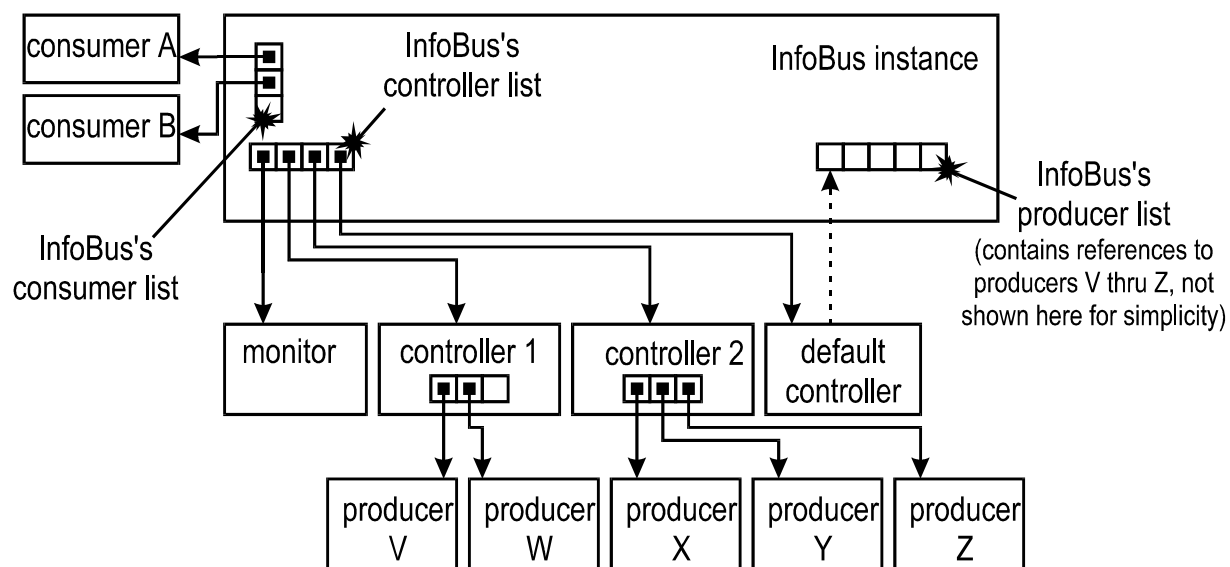


Figure 7-1: Event distribution example illustrating multiple controllers

Suppose Consumer A requests an item called “Sales.” It does this by calling the InfoBus method `findDataItem()`. The InfoBus handles the request by calling each controller in order of priority.

The InfoBus calls `findDataItem()` on each controller in turn until one says the event has been handled. The monitor controller logs the event. Controller 1 asks producers V and W by calling `findDataItem()` with its Vector as the list of targets. In this example neither V nor W can supply a “Sales” item, so Controller 1 puts nothing in the `foundItem` Vector and returns `false`. The InfoBus next calls Controller 2, which calls producer X. X is able to supply the item, so Controller 2 stores the item reference in the `foundItem` Vector, does not need to call its other producers, and returns `true`. The default controller is not called in this case.

As another example, suppose producer V announces the availability of “Sales Forecast.” Each controller is invited in turn to handle the event. Because controllers 1 and 2 only handle data item requests, the default controller handles the announcement by distributing it to both consumers.

8. Policy helper

The `InfoBusPolicyHelper` interface provides a means of centralizing security-related decisions. The decisions made in this class implement the permission and security mechanism used for all consumers, producers, and data controllers.

The `InfoBus` class holds a single object of type `InfoBusPolicyHelper` as a static variable. Whenever an `InfoBus` object is about to perform an activity (such as registering an `InfoBusConsumer` or distributing an `InfoBusItemAvailableEvent`), it first calls a method on that static variable to ensure the policy helper permits it. If the policy helper does not approve of the action, it throws a runtime exception which goes uncaught in the `InfoBus`.

This design strategy is optimized for use of the Java 2 Platform Security Architecture — e.g., `AccessController.checkPermission()` is called within the policy helper and an `AccessControlException` thrown if the action is not permitted. However, the specification of `InfoBusPolicyHelper` is general enough to allow implementations which do not rely on Java 2 Platform mechanisms.

The other activity delegated to the `InfoBusPolicyHelper` is the creation of default `InfoBus` names — when an `InfoBusMember` calls `InfoBus.get(Object)`, the `InfoBus` in turn calls the policy helper's `generateDefaultName()` method.

Security and default naming functions are encapsulated in `InfoBusPolicyHelper` to provide flexibility in making these decisions. The `javax.infobus` package includes a default implementation of the interface in the `javax.infobus.DefaultPolicy` class, but specification of a different policy helper class can be made by setting a system property called `javax.infobus.InfoBusPolicy`.

8.1 The `InfoBusPolicyHelper` interface

The `InfoBusPolicyHelper` interface encapsulates several security decisions and default `InfoBus` name generation in one interface. The `InfoBus` class holds a single object of this type as a static variable, and all `InfoBus` instances perform a call to one of its security methods before performing an action. The implementation of `InfoBusPolicyHelper` considers the action being requested on behalf of a caller and may throw a runtime exception if it disapproves. This consideration may include examining the call stack and deciding on the basis of the classes it finds whether to grant permission.

It is not necessary for the policy helper to pass judgement on every method provided in the interface: a very relaxed policy helper may implement all security checks as no-ops (empty methods), while a very strict policy helper may introduce an arbitrarily complex set of checks and permissions before approving any action.

The static variable holding the `InfoBusPolicyHelper` in use is initialized when an `InfoBus` static method is called or when an `InfoBus` constructor is called, whichever occurs first. Once instantiated the policy helper static variable is immutable: no means of changing the policy helper is available short of restarting the JVM.

```
public String generateDefaultName(Object object)
```

The implementer of `InfoBusPolicyHelper` is responsible for determining the default `InfoBus` naming strategy in use. The `InfoBus` limits the parameter of the `get()` method to an object of type `java.awt.Component` or `java.beans.BeansContext`. From the *object* parameter, the policy helper must create a `String` that denotes the default `InfoBus` name for the object.

A default name policy must generate names that allow objects in a shared space — for example, on a single web page or within a single `BeanContext` — to communicate without having prior knowledge of what `InfoBus` name to specify. Ideally, the default name policy should generate names in which objects in other spaces (i.e. a different web page) get a different bus, so that `InfoBuses` do not get overpopulated.

```
public void canGet(String busName)
```

```
public void canJoin(InfoBus infobus, InfoBusMember member)
```

```
public void canRegister(InfoBus infobus, InfoBusMember member)
```

```
public void canPropertyChange(InfoBus infobus,  
    java.beans.PropertyChangeEvent event)
```

```

public void canAddDataProducer(InfoBus infobus, InfoBusDataProducer producer)
public void canAddDataConsumer(InfoBus infobus, InfoBusDataConsumer consumer)
public void canAddDataController(InfoBus infobus, InfoBusDataController controller,
    int priority)
public void canFireItemAvailable(InfoBus infobus, String dataItemName,
    InfoBusDataProducer producer)
public void canFireItemRevoked(InfoBus infobus, String dataItemName,
    InfoBusDataProducer producer)
public void canRequestItem(InfoBus infobus, String dataItemName,
    InfoBusDataConsumer consumer)

```

The security methods on InfoBusPolicyHelper are named to reflect the methods from which an InfoBus object will call them, and usually include as parameters a reference to the calling InfoBus and all parameters that the InfoBus method has been provided. In general, the policy helper method canXYZ is called by an InfoBus before performing the activity in method xyz. For example, the implementation of the InfoBus method addDataProducer(InfoBusDataProducer *producer*) calls the InfoBusPolicyHelper method canAddDataProducer(InfoBus, InfoBusDataProducer)—with parameters set to (*this*, *producer*)—before permitting the producer’s registration. The exception to this naming pattern is InfoBusPolicyHelper.canRequestItem(), which is called by the InfoBus methods findDataItem() and findMultipleDataItems().

The parameters provided to the InfoBusPolicyHelper method generally include all those provided to the parent InfoBus method to allow maximum flexibility in determining whether an action is permissible. Note that an InfoBusPolicyHelper implementation based on Java 2 Platform security mechanisms may find one or more parameters unnecessary.

These security methods are called from InfoBus before performing an action. InfoBus methods that check to see if the InfoBus object is stale will perform that check and throw an Exception if it is stale before calling the policy helper security method (because the exception prevents the action, and a replacement bus may permit the action where the stale one may not have).

The InfoBusPolicyHelper implementation in use may consider the requested action and throw a RuntimeException if it disapproves; if the action is permitted, the security method simply returns. This technique lends itself to use of the Java 2 Platform security mechanisms: the policy helper implementation can introduce a set of Permissions corresponding to InfoBus activity that it may want to restrict. When the security method is called, the policy helper can formulate the appropriate permission and call AccessController.checkPermission() to see if the Permission is granted in the current java.policy. If it is not, the AccessController throws an AccessControlException. Both the policy helper and the InfoBus then propagate the AccessControlException, which should generally propagate to the system level to indicate the unpermitted activity.

8.2 The DefaultPolicy class

This class implements the InfoBusPolicyHelper interface and is the policy helper put into effect if the javax.infobus.InfoBusPolicy system property is nonexistent or unreadable. For the InfoBus 1.1 release, which supports JDK 1.1, the DefaultPolicy class:

- generates the default InfoBus name based on DOCBASE from an AppletContext.
- performs no security checks, because Java 2 Platform is required for all security mechanisms. All of the methods defined for the InfoBusPolicyHelper interface have empty implementations for the DefaultPolicy class in InfoBus 1.1, and will not throw an exception if called. This simulates the behavior of passing a permission check in InfoBus 2.0.

InfoBus 2.0 will include a new implementation of this class that uses the Java 2 Platform security features to do its work.

9. Exceptions

9.1 InfoBusMembershipException

This Exception is thrown by the InfoBus core code when it will not allow an action related to membership with the bus. For example, this exception is thrown when `InfoBusMemberSupport.joinInfoBus()` is called on a class which is already a member of a bus. It is also thrown when `InfoBusBeanSupport.setInfoBusName()` is called, but a `PropertyVetoException` is thrown by the resulting `setInfoBus()` call to change the bus membership.

9.2 DuplicateColumnException

This exception is thrown in `RowsetAccess` methods when a duplicate column is found.

9.3 ColumnNotFoundException

This exception is thrown in `RowsetAccess` methods when a specified column cannot be found.

9.4 InvalidDataException

`InvalidDataException` extends `java.lang.Exception`, and is thrown by any InfoBus method which modifies data. The producer may throw the exception when data cannot be accepted as specified, such as an invalid date.

9.5 RowsetValidationException

`RowsetValidationException` extends `InvalidDataException` to provide more information about the Rowset on which the exception occurred. A `RowsetValidationException` may be thrown by any `RowsetAccess` (or sub-interface) method which modifies data.

```
public RowsetValidationException(String message, RowsetAccess rowset,  
                                InfoBusPropertyMap map)
```

The constructor for `RowsetValidationException` specifies a *message* string, a reference to the *rowset* on which validation failed, and an optional reference to an implementation of `InfoBusPropertyMap` for supporting properties on this exception. When properties are not supported, `null` should be specified for *map*.

```
public RowsetAccess getRowset()
```

This method returns a reference to the `RowsetAccess` object on which the validation problem was detected.

```
public Object getProperty(String propertyName)
```

Returns a property or metadata information about the validation exception. Support for properties is optional; `null` should be returned for unsupported properties.

Support for properties is optional. If no property map was specified on the constructor, `getProperty()` returns `null`. Otherwise, this method calls `InfoBusPropertyMap.get()` with the specified *key* and returns the result. `null` is the conventional return value when the specified *key* is not supported as a property name.

9.6 UnsupportedOperationException

This runtime exception is temporarily defined in `javax.infobus` for the InfoBus 1.1 release for use with JDK 1.1. Applications built with Java 2 Platform should use `java.lang.UnsupportedOperationException` instead. This method may be thrown for optional methods that are not supported, such as methods that modify data when the producer provides read-only access.

9.7 StaleInfoBusException

This `RuntimeException` is thrown when certain operations are attempted on an InfoBus instance which has become “stale.” See Chapter 2 for an explanation of how stale InfoBus instances occur, and how to avoid them.

10. Guidelines for well-behaved InfoBus components

This section offers guidelines for developers creating InfoBus components that will make them generally useful with other components.

10.1 Membership

For maximum flexibility, applets should accept a bus name as a parameter in the HTML tags, and use it if found. If none is found, it should use the default InfoBus. InfoBus members should be prepared to have their “InfoBus” property changed by an outside class (such as a container). The InfoBusMemberSupport class provides this support.

10.2 Data items

Specifying names

It should be possible to name data items by way of applet parameters, the UI, or both.

Rendezvous

All items announced as available should have a matching revoked announcement. In addition to sending an InfoBusRevokedEvent, the revoked change event should be sent to data item change listeners.

Interfaces to be implemented for data items

Though not required by the API, an InfoBus-compliant data item should always implement the DataItem interface and support all methods defined there by returning non-null objects as required. The data item name should be the same as the one specified in the available or request event. We recommend that sub-items of a collection data items also implement this interface.

Data producers should provide as many standard access interfaces as makes sense to support a wide variety of consumers. The standard access interfaces include those defined in chapters 4 and 5 of this specification, along with the collection interfaces defined by Java 2 Platform.

Data consumers should always call DataItem.release() when they are dropping their last reference to a data item they have requested, and should never issue any other calls on the data item after calling release().

Lists of fields and contents

A common form of exchange is the fieldname-value pair. This form is consistent with the notion of passing bags of properties. It is anticipated that this kind of table of values will be a common exchange medium. Note that a list of fields and their contents is an appropriate way to model (with careful implementation) a single row of a database table.

As such, it is an appropriate medium for applications like forms fill-out, or data browsing. In addition to the passing of repeating tuples of data, the field/value pair is also a good way to pass general parameter information from producer to consumer. Field/value lists should be modeled as a Map interface (from the JDK Collections).

Streams of data

InputStreams are considered single-valued items and could be presented in an ImmediateAccess object. However, the intent of the InfoBus is to impose structure on the data, in an effort to make the data comprehensible to the widest audience of consumers. In this sense, offering only an InputStream where more significant formatting is possible is a shortcut with potentially expensive future consequences. In general, streams of data should be handled via the Transferable mechanism as per the AWT clipboard specifications.

Of course, some types of InputStreams, like multimedia streams that are “played” by the recipient, will not lend themselves to such formatting.

Appendix A: Changes in the InfoBus 1.2 Specification

This section lists changes that have been made in the specification, comparing this version to the previous version (InfoBus 1.1.1). The changes are organized as “Major Changes” (those that add or deprecate interfaces or classes, deprecate interfaces, or otherwise imply changes in the way things should work) and “Minor Changes” (clarifications, corrections to typos, corrections to the API where the spec was wrong but the code and JavaDoc were right, and so on).

The essential change for InfoBus 1.2 is the addition of a new access interface, called `ReshapeableArrayAccess`, that extends `ArrayAccess` to add an API for allowing the consumer to insert and delete rows or columns, and change dimensions on the array at will. There are a number of supporting changes, mainly in announcing changes that are possible with the new interface.

Major Changes in the InfoBus 1.2 Specification

- In section 4.12, “The `ArrayAccess` interface”, the method `getItemByCoordinate()` now specifies that null is returned to indicate an empty cell.
- In section 3.6, “DataFlavors and describing data items”, the MIME string constants have been redefined as a bug fix so that applications that use DataFlavors with the defined MIME strings, and are built with JDK 1.1.x and InfoBus 1.2 will also work on Java Platform 2. The MIME strings introduced in the earlier spec were missing the “infobus.javax.” qualification before the interface name, and that caused applications to fail when used on the Java 2 Platform.
- Section 4.13, “The `ReshapeableArrayAccess` interface”, describes a new access interface that was introduced in this release.
- Section 6.1, “The `DataItemChangeManager` interface”, adds a requirement that data items that implement `ReshapeableArrayAccess` must also implement `DataItemChangeManager`, as consumers of this interface typically cannot function well, without the `DataItemShapeChangedEvent` notification.
- Section 6.2 was renamed to “The `DataItemChangeManagerSupport` class”. The old class (`DataItemChangeSupport`) has been deprecated, and is replaced by the new class (`DataItemChangeManagerSupport`). The extra word, “Manager”, was added to the class name to distinguish it from another new class, `DataItemChangeListenerSupport`. `DataItemChangeManagerSupport` has one new method (to fire `DataItemShapeChangedEvents` to consumers of a `ReshapeableArrayAccess` item) compared to the older class it replaces.
- Section 6.4, “The `DataItemShapeChangeListener` interface”, describes a new interface that consumers of the `ReshapeableArrayAccess` will use instead of `DataItemChangeListener`.
- Section 6.5, “The `DataItemChangeListenerSupport` class”, describes a new class that makes it easier to implement data item change listeners (of either type).
- Section 6.6, “The `DataItemChangeEvent` class and event subclasses”, has a subsection “*DataItemValueChangeEvent, DataItemRevokedEvent and RowsetCursorMovedEvent classes*” is renamed to “*DataItemValueChangeEvent, DataItemRevokedEvent, DataItemShapeChangedEvent and RowsetCursorMovedEvent classes*”, since it also describes the new change event. Figure 6-2 is also revised to include the new event.
- Section 6.7, “Distribution and handling of `DataItemChangeEvents`”, has a new paragraph describing how the `DataItemDeletedEvent` is used for data items that implement the `ReshapeableArrayAccess` interface, as well as a description of how the new `DataItemShapeChangedEvent` is used for those data items.
- Section 6.8, “Examples of event propagation”, adds examples that apply to a `ReshapeableArrayAccess` data item.
- A few classes, interfaces, and methods are deprecated. These are described in Appendix B. Appendix B will accumulate all deprecated classes, interfaces, and methods in future releases.

Minor Changes in the InfoBus 1.2 Specification

The following minor changes have been made in this version of the specification, mainly fixes to spec wording problems. These do not represent any changes to the functioning of InfoBus:

- Some sections are renumbered because of new sections inserted before them.
- In several places, references to InfoBus 1.2 have been changed to InfoBus 2.0. The planned release that uses Java 2 Platform security features, BeanContext, and the like, has been renamed from InfoBus 1.2 to InfoBus 2.0. Additional releases of InfoBus 1.x for JDK 1.1.x will be numbered 1.3 and so on.
- In section 2.8, “The InfoBus class”, the incorrect reference to “InfoBusDefaultPolicies” has been corrected to “DefaultPolicy.”
- In section 3.2, “Event listeners,” the spec used to state that methods to add event listeners were called “in” the start() method. This has been corrected to “during the execution of” the start() method. The same correction was applied for removing event listeners during the execution of the stop() method. The changes reflects the fact that in the sample application, the event listeners are added and removed in the propertyChange() method of the PropertyChangeListener interface.
- Sections 4.2 and 4.3 used to refer to the now-obsolete clock and timesource sample apps. They has been reworked to describe the newer SimpleProducerBean and MonetaryDataItem example.
- In section 4.11, “The ImmediateAccess interface”, the API for ImmediateAccess.setValue() incorrectly specified Object as the return type. This has been corrected to void, to agree with the code and JavaDoc.
- In section 5.4, “The RowsetAccess interface”, a wording error was fixed.
- In section 6.2, “The DataItemChangeManagerSupport class”, the descriptions for methods that fire events now include a description of the propertyMap parameter. This description was missing in the last spec.
- In section 6.4, “The DataItemChangeEvent class and event subclasses”, a wording error was fixed.
- In section 6.5, “Distribution and handling of DataItemChangeEvents”, for the subsection called DataItemValueChangedEvent, the obsolete TimeSource sample app was used as an example. It has been reworked to remove that reference.
- In section 8.1, “The InfoBusPolicyHelper interface”, the text ran over the bottom of the page. The layout has been fixed.

Appendix B - Deprecated Interfaces, Classes and Methods

This appendix lists various interfaces, classes, and/or methods that used to be a part of earlier InfoBus specifications and are still present in the InfoBus.jar file to support existing applications, but are marked deprecated because they are not the preferred means of coding InfoBus applications from this release onward.

B.1 DataItemChangeSupport replaced by DataItemChangeManagerSupport

The DataItemChangeSupport class provided an implementation of the DataItemChangeListener interface for InfoBus 1.1.1 and earlier. In InfoBus 1.2 and onward, the preferred support class for DataItemChangeListener is now called DataItemChangeManagerSupport. The older DataItemChangeSupport class is deprecated. The newer DataItemChangeManagerSupport contains all of the methods from DataItemChangeSupport, and adds a new method for firing the new DataItemShapeChangedEvent.

The reason for renaming the class is that we have also introduced a new support class for DataItemChangeListener, called DataItemChangeListenerSupport, and we thought it would be confusing if the support class for the change manager did not have the word “Manager” in it to distinguish it from the “Listener” support class.

* * * END OF DOCUMENT * * *

